💼 Resume + Job Description RAG Analyzer

Development order (blank pages are not counted.)

We'll build and test it exactly like your previous AI Study Assistant:

Step 1: `document_loader.py`

Resume + JD → extracted text

Step 2: `chunking.py`

Text → meaningful chunks

Step 3: `embeddings.py`

Chunks → 384-dimensional embeddings

Step 4: `vector_store.py`

Embeddings → FAISS

Step 5: `retriever.py`

Question → relevant Resume/JD chunks

Step 6: `llm.py`

Context + question → Gemini analysis

Step 7: `rag_pipeline.py`

Connect Retriever + LLM

Step 8: `app.py`

Flask API

Step 9: Frontend

Resume upload + JD upload + analysis dashboard

Step 10: Final features

- Skill matching
- Missing skills
- Relevant projects
- Resume improvement suggestions
- JD-based interview questions
- Technical questions
- Section relevance explanations
- Sources/citations

💼 Project: Resume + Job Description RAG Analyzer

Final workflow

```
Resume.pdf + Job_Description.pdf
                ↓
         Document Parsing
                ↓
           Text Extraction
                ↓
            Chunking
                ↓
           Embeddings
                ↓
         FAISS Vector DB
                ↓
         Relevant Retrieval
                ↓
             RAG
                ↓
           Gemini
                ↓
       Personalized Analysis
```

```
            ↓
      React Frontend
```

The important part is that the system should **not simply compare two PDFs with string matching**. It should retrieve relevant resume/JD information and give Gemini the retrieved context to produce the analysis.

STEP 1 — Create the project

Create:

```
Resume-JD-RAG-Analyzer/
│
├── Backend/
│
└── Frontend/
```

Backend:

```
Backend/
├── documents/
├── vector_store/
├── src/
├── app.py
├── .env
└── requirements.txt
```

Frontend:

```
Frontend/
├── public/
├── src/
├── package.json
├── vite.config.js
└── index.html
```

STEP 2 — Backend environment

Create your virtual environment:

```
Backend/
└── venv/
```

Activate it.

Then install the main packages:

```
flask
flask-cors
python-dotenv
pymupdf
sentence-transformers
faiss-cpu
google-genai
numpy
```

You can add other packages later if a feature requires them.

STEP 3 — Create `.env`

Inside:

```
Backend/.env
```

put:

```
GEMINI_API_KEY=YOUR_API_KEY
```

Don't put the actual API key into GitHub.

Add:

```
.env
venv/
```

```
__pycache__/
vector_store/
```

to `.gitignore`.

STEP 4 — Build document loader

Create:

`Backend/src/file_loader.py`

Purpose:

```
Resume.pdf
     ↓
PDF text extraction

Job_Description.pdf
     ↓
PDF text extraction
```

The loader should extract:

```
text
source
page
document_type
```

For example:

```
{
    "text": "...",
    "source": "Resume.pdf",
    "page": 1,
    "document_type": "resume"
}
```

and:

```
{
    "text": "...",
    "source": "Job_Description.pdf",
    "page": 1,
    "document_type": "job_description"
}
```

Test

Run:

```
python src/file_loader.py
```

Expected:

```
========================================
DOCUMENT LOADER TEST
========================================

Resume loaded successfully.
Job Description loaded successfully.

Resume pages: 2
JD pages: 1
```

If your files have names such as:

Plain text

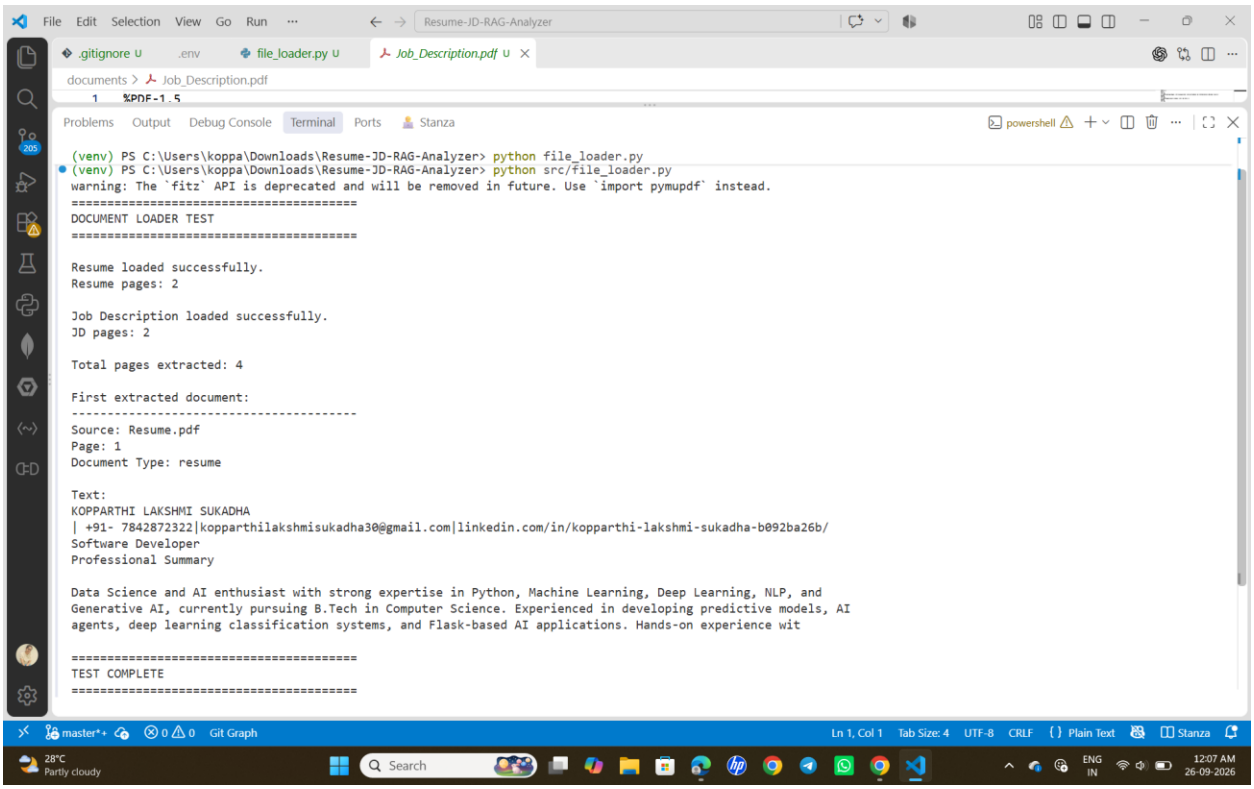
```
resume (1).pdf
my_resume.pdf
Wipro_JD.pdf
```

either rename them to:

Plain text

```
Resume.pdf
Job_Description.pdf
```

or tell me the exact filenames and I'll modify the code.



STEP 5 — Build chunking

Create:

`Backend/src/chunking.py`

Flow:

```
Extracted Documents
        ↓
      Text
        ↓
     Chunks
```

Don't make chunks too large.

For example:

```
chunk_size = 500
overlap = 100
```

Each chunk should preserve its metadata:

```
{
    "text": "...",
    "source": "Resume.pdf",
    "page": 1,
    "document_type": "resume"
}
```

Why metadata matters

Later Gemini can say:

```
Resume.pdf — Page 1
Job_Description.pdf — Page 2
```

instead of giving an answer without a source.

```
(venv) PS C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer> python src/chunking.py
f it has the same name as a library you intended to import)
(venv) PS C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer> python src/chunking.py
warning: The `fitz` API is deprecated and will be removed in future. Use `import pymupdf` instead.
==================================================
CHUNKING TEST
==================================================

Documents extracted: 4
Total chunks: 22

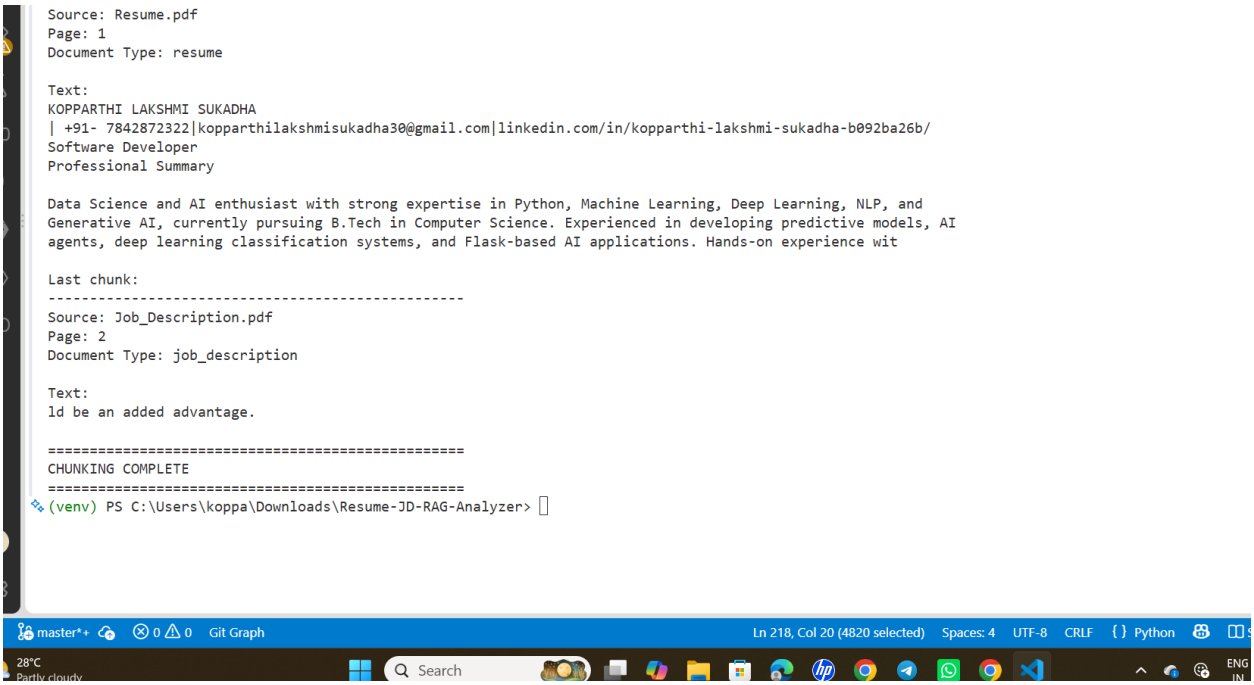
Chunk size: 500
Chunk overlap: 100

First chunk:
--------------------------------------------------
Source: Resume.pdf
Page: 1
Document Type: resume

Text:
KOPPARTHI LAKSHMI SUKADHA
| +91- 7842872322|kopparthilakshmisukadha30@gmail.com|linkedin.com/in/kopparthi-lakshmi-sukadha-b092ba26b/
Software Developer
Professional Summary

Data Science and AI enthusiast with strong expertise in Python, Machine Learning, Deep Learning, NLP, and
Generative AI, currently pursuing B.Tech in Computer Science. Experienced in developing predictive models, AI
agents, deep learning classification systems, and Flask-based AI applications. Hands-on experience wit

Last chunk:
--------------------------------------------------
Source: Job_Description.pdf
Page: 2
```



STEP 6 — Build embeddings

Create:

`Backend/src/embeddings.py`

Use:

`all-MiniLM-L6-v2`

Flow:

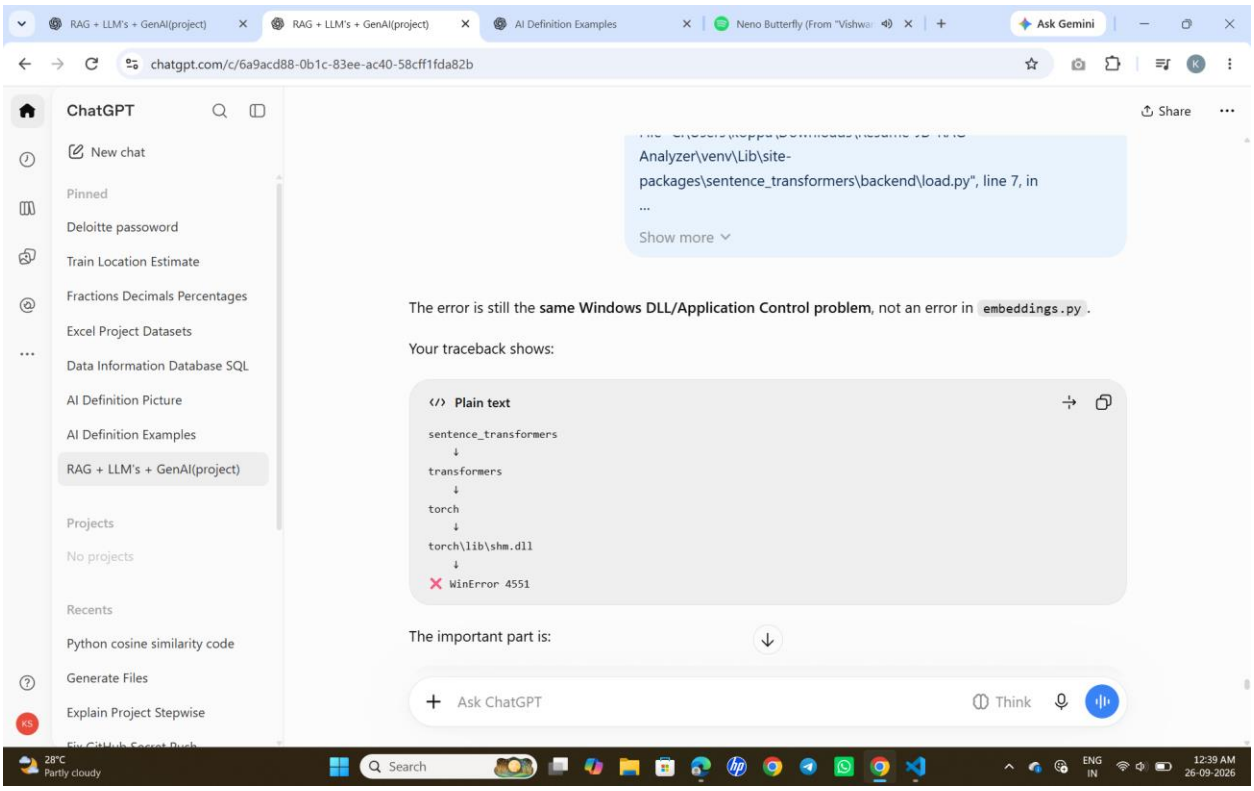
```
Chunks
   ↓
Sentence Transformer
   ↓
384-dimensional vectors
```

For example:

```
Resume chunk
      ↓
[0.012, -0.031, 0.224, ...]
```

The same embedding model must be used for:

- resume chunks
- JD chunks
- user questions



The important part is:

```
Plain text
OSError: [WinError 4551]
An Application Control policy has blocked this file.
```

Also, I would **not blame Python 3.14 itself** here. Current PyTorch Windows documentation lists Python 3.10–3.14 as supported, and current Sentence Transformers recommends Python 3.10+. PyTorch +1

Do this first

Your virtual environment is inside:

```
Plain text
C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer\venv
```

Since Windows is specifically blocking a native DLL, let's test with a **new environment outside Downloads**.

Run these commands from PowerShell:

```
powershell
deactivate
```

Then:

```
powershell
mkdir C:\venvs
```

Then:

```powershell
mkdir C:\venvs
```

Create the environment:

```powershell
py -3.12 -m venv C:\venvs\resume-rag
```

Activate it:

```powershell
C:\venvs\resume-rag\Scripts\Activate.ps1
```

Check:

```powershell
python --version
```

You should see:

```Plain text
Python 3.12.x
```

Then install PyTorch CPU:

Plain text

```
Python 3.12.x
```

Then install PyTorch CPU:

powershell

```
python -m pip install --upgrade pip
```

powershell

```
pip install torch --index-url https://download.pytorch.org/whl/cpu
```

Test **only PyTorch first**:

powershell

```
python -c "import torch; print('PyTorch OK:', torch.__version__)"
```

If PyTorch works

Then install Sentence Transformers:

powershell

```
pip install sentence-transformers
```

Test:

powershell

```
python -c "import sentence_transformers; print('Sentence Transformers OK')"
```

Ask ChatGPT Think

powershell

```
python -c "import sentence_transformers; print('Sentence Transformers OK')"
```

Then go to your project:

powershell

```
cd C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer
```

Because your activated environment is `C:\venvs\resume-rag`, your project will still use that environment.

Run:

powershell

```
python src/embeddings.py
```

You want to reach:

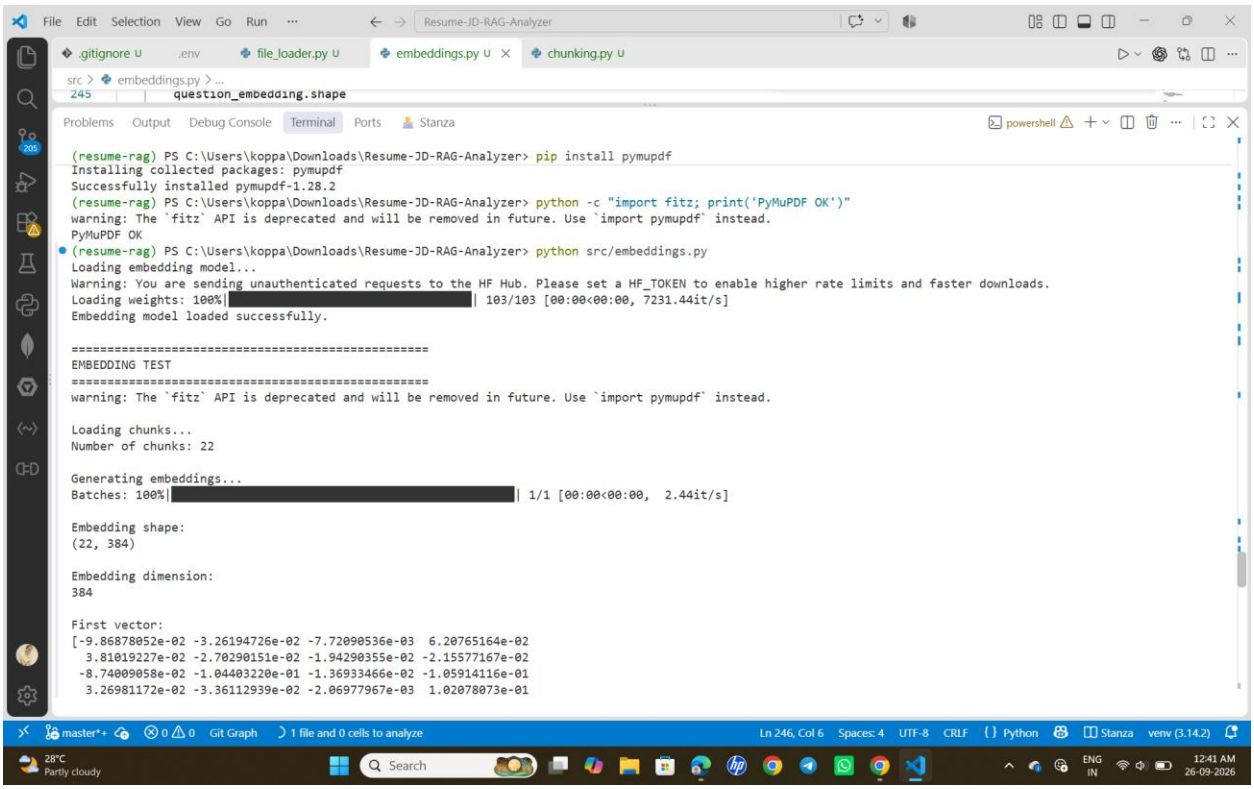
Plain text

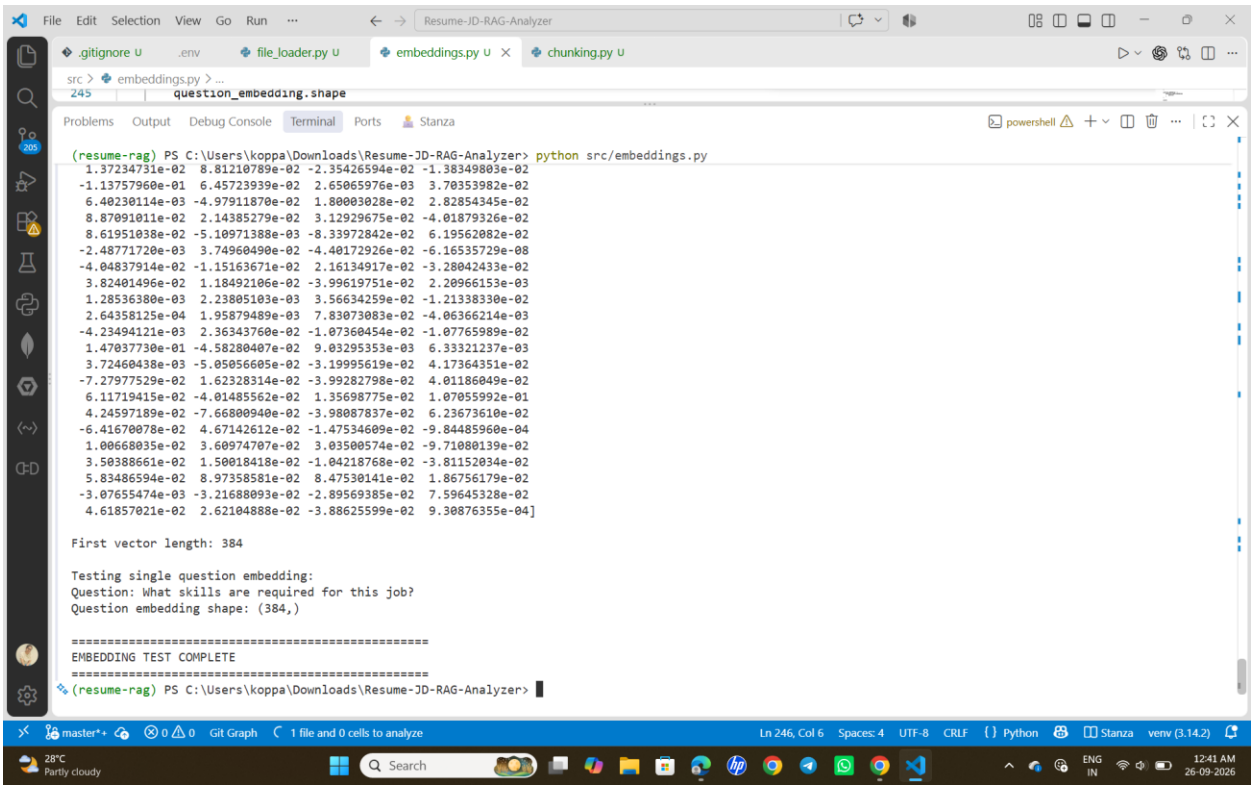
```
Embedding model loaded successfully.

Number of chunks: ...

Embedding shape:
(..., 384)

Embedding dimension:
384
```


File Edit Selection View Go Run
Resume-JD-RAG-Analyzer
.gitignore U
.env
file_loader.py U
embeddings.py U
chunking.py U
src > embeddings.py > ...
245 question_embedding.shape
Problems Output Debug Console Terminal Ports Stanza
powershell
(resume-rag) PS C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer> pip install pymupdf
Installing collected packages: pymupdf
Successfully installed pymupdf-1.28.2
(resume-rag) PS C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer> python -c "import fitz; print('PyMuPDF OK')"
warning: The `fitz` API is deprecated and will be removed in future. Use `import pymupdf` instead.
PyMuPDF OK
(resume-rag) PS C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer> python src/embeddings.py
Loading embedding model...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%| 103/103 [00:00<00:00, 7231.44it/s]
Embedding model loaded successfully.
==
EMBEDDING TEST
==
warning: The `fitz` API is deprecated and will be removed in future. Use `import pymupdf` instead.
Loading chunks...
Number of chunks: 22
Generating embeddings...
Batches: 100%| 1/1 [00:00<00:00, 2.44it/s]
Embedding shape:
(22, 384)
Embedding dimension:
384
First vector:
[-9.86878052e-02 -3.26194726e-02 -7.72090536e-03 6.20765164e-02
3.81019227e-02 -2.70290151e-02 -1.94290355e-02 -2.15577167e-02
-8.74009058e-02 -1.04403220e-01 -1.36933466e-02 -1.05914116e-01
3.26981172e-02 -3.36112939e-02 -2.06977967e-03 1.02078073e-01
master*+ 0 0 Git Graph 1 file and 0 cells to analyze
Ln 246, Col 6 Spaces: 4 UTF-8 CRLF Python Stanza venv (3.14.2)
28°C Partly cloudy
Search
ENG IN
12:41 AM 26-09-2026


File Edit Selection View Go Run
Resume-JD-RAG-Analyzer
.gitignore U
.env
file_loader.py U
embeddings.py U
chunking.py U
src > embeddings.py > ...
245 question_embedding.shape
Problems Output Debug Console Terminal Ports Stanza
powershell
(resume-rag) PS C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer> python src/embeddings.py
1.37234731e-02 8.81210789e-02 -2.35426594e-02 -1.38349803e-02
-1.13757960e-01 6.45723939e-02 2.65065976e-03 3.70353982e-02
6.40230114e-03 -4.97911870e-02 1.80003028e-02 2.82854345e-02
8.87091011e-02 2.14385279e-02 3.12929675e-02 -4.01879326e-02
8.61951038e-02 -5.10971388e-03 -8.33972842e-02 6.19562082e-02
-2.48771720e-03 3.74960490e-02 -4.40172926e-02 -6.16535729e-08
-4.04837914e-02 -1.15163671e-02 2.16134917e-02 -3.28042433e-02
3.82401496e-02 1.18492106e-02 -3.99619751e-02 2.20966153e-03
1.28536380e-03 2.23805103e-03 3.56634259e-02 -1.21338330e-02
2.64358125e-04 1.95879489e-03 7.83073083e-02 -4.06366214e-03
-4.23494121e-03 2.36343760e-02 -1.07360454e-02 -1.07765989e-02
1.47037730e-01 -4.58280407e-02 9.03295353e-03 6.33321237e-03
3.72460438e-03 -5.05056605e-02 -3.19995619e-02 4.17364351e-02
-7.27977529e-02 1.62328314e-02 -3.99282798e-02 4.01186049e-02
6.11719415e-02 -4.01485562e-02 1.35698775e-02 1.07055992e-01
4.24597189e-02 -7.66800940e-02 -3.98087837e-02 6.23673610e-02
-6.41670078e-02 4.67142612e-02 -1.47534609e-02 -9.84485960e-04
1.00668035e-02 3.60974707e-02 3.03500574e-02 -9.71080139e-02
3.50388661e-02 1.50018418e-02 -1.04218768e-02 -3.81152034e-02
5.83486594e-02 8.97358581e-02 8.47530141e-02 1.86756179e-02
-3.07655474e-03 -3.21688093e-02 -2.89569385e-02 7.59645328e-02
4.61857021e-02 2.62104888e-02 -3.88625599e-02 9.30876355e-04]
First vector length: 384
Testing single question embedding:
Question: What skills are required for this job?
Question embedding shape: (384,)
==
EMBEDDING TEST COMPLETE
==
(resume-rag) PS C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer>
master*+ 0 0 Git Graph 1 file and 0 cells to analyze
Ln 246, Col 6 Spaces: 4 UTF-8 CRLF Python Stanza venv (3.14.2)
28°C Partly cloudy
Search
ENG IN
12:41 AM 26-09-2026

STEP 7 — Build FAISS vector store

Create:

```
Backend/src/vector_store.py
```

Flow:

```
Resume chunks
      +
JD chunks
      ↓
  Embeddings
      ↓
     FAISS
```

Save:

```
Backend/
└── vector_store/
    ├── index.faiss
    └── metadata.pkl
```

Metadata should contain:

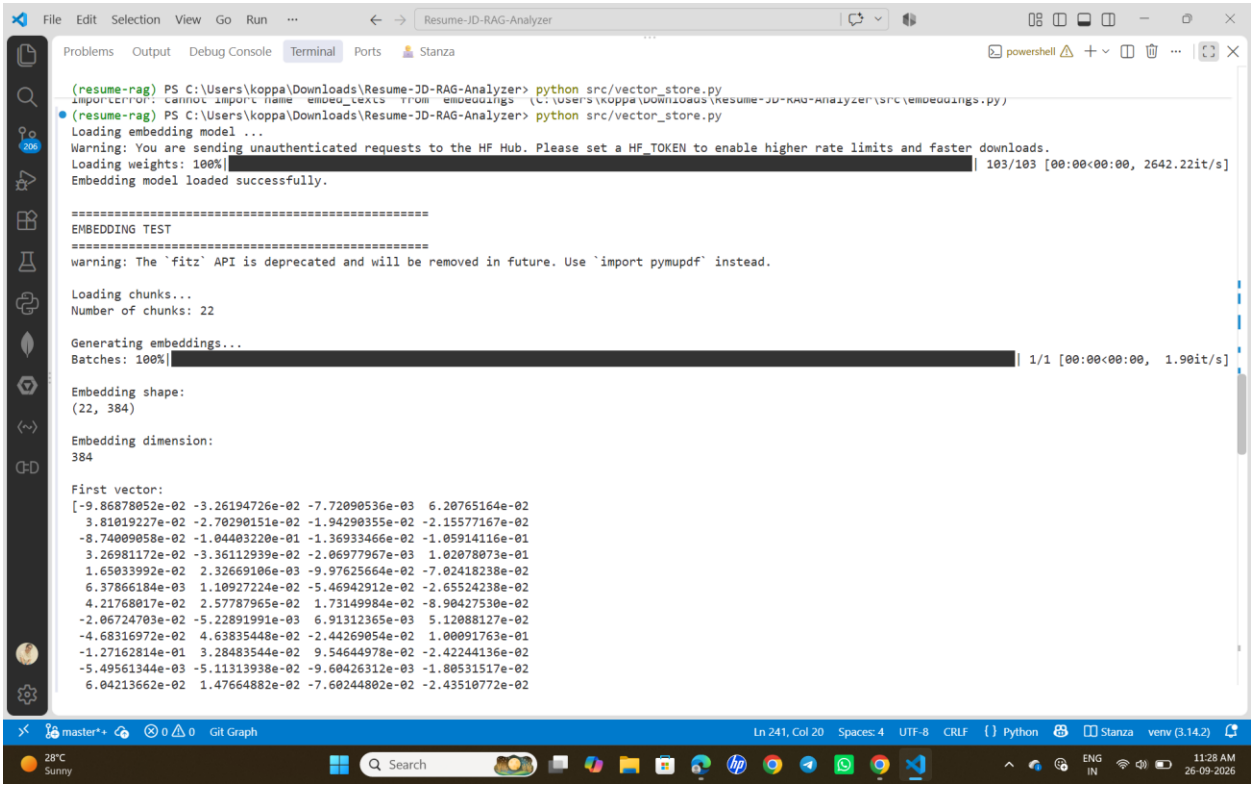
```
text
source
page
document_type
```

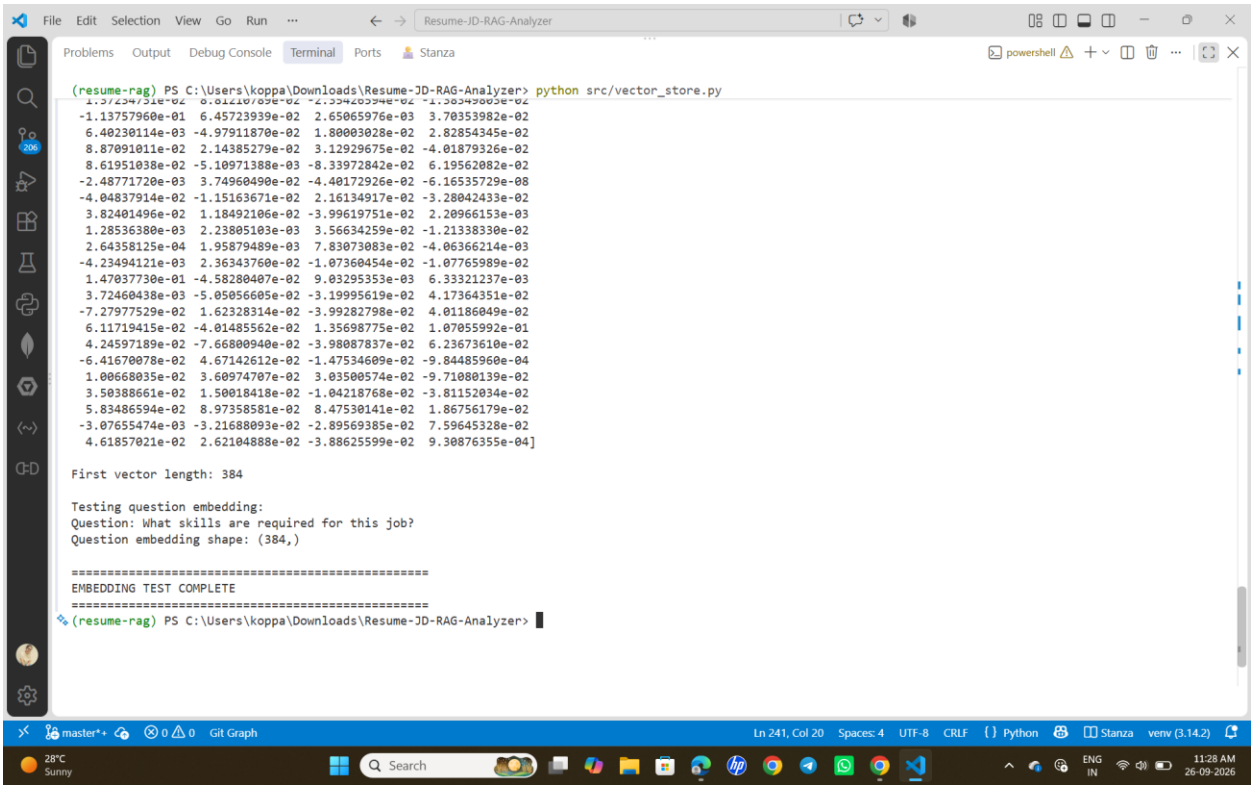
Example:

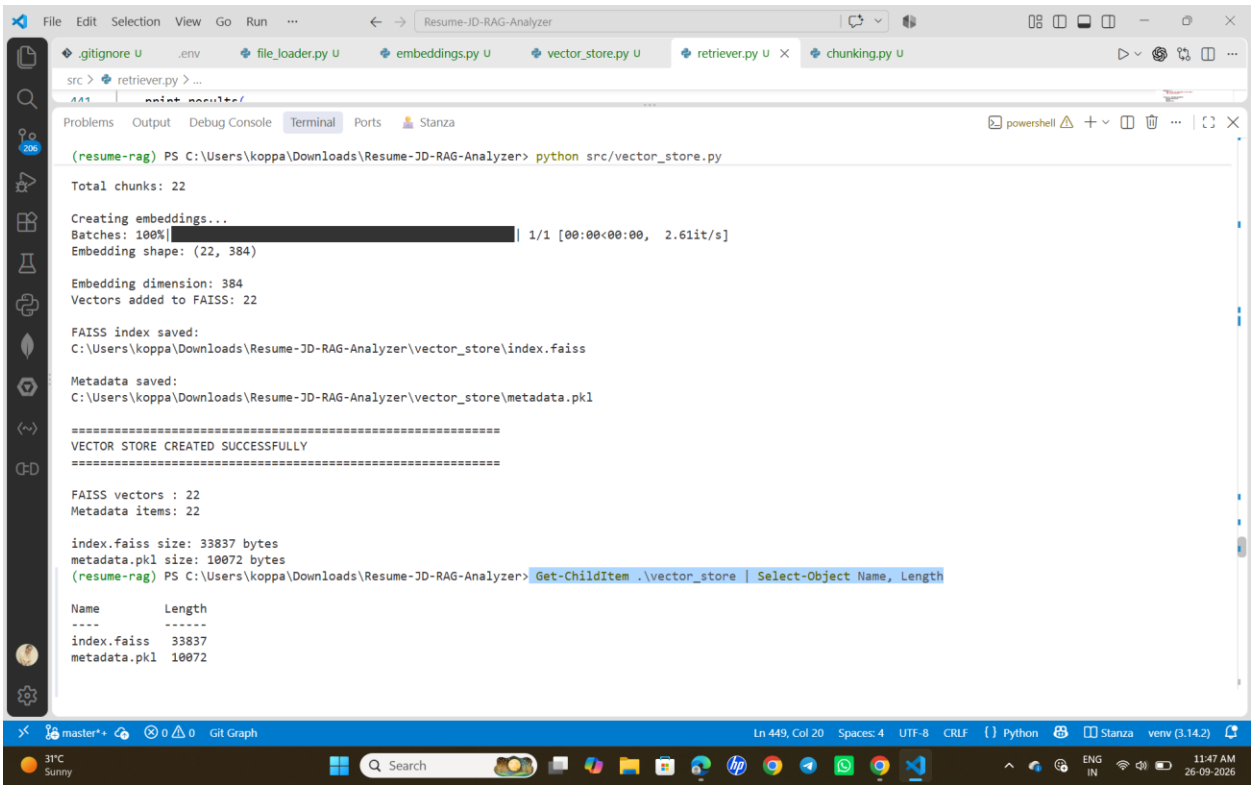
```
Resume.pdf
Page 2
document_type = resume
```

and:

```
Job_Description.pdf
Page 1
document_type = job_description
```


File Edit Selection View Go Run
Resume-JD-RAG-Analyzer
Problems Output Debug Console Terminal Ports Stanza
powershell
(resume-rag) PS C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer> python src/vector_store.py
ImportError: cannot import name 'embed_texts' from 'embeddings' (C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer\src\embeddings.py)
(resume-rag) PS C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer> python src/vector_store.py
Loading embedding model ...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%| 103/103 [00:00<00:00, 2642.22it/s]
Embedding model loaded successfully.
==
EMBEDDING TEST
==
warning: The `fitz` API is deprecated and will be removed in future. Use `import pymupdf` instead.
Loading chunks...
Number of chunks: 22
Generating embeddings...
Batches: 100%| 1/1 [00:00<00:00, 1.90it/s]
Embedding shape:
(22, 384)
Embedding dimension:
384
First vector:
[-9.86878052e-02 -3.26194726e-02 -7.72090536e-03 6.20765164e-02
3.81019227e-02 -2.70290151e-02 -1.94290355e-02 -2.15577167e-02
-8.74009058e-02 -1.04403220e-01 -1.36933466e-02 -1.05914116e-01
3.26981172e-02 -3.36112939e-02 -2.06977967e-03 1.02078073e-01
1.65033992e-02 2.32669106e-03 -9.97625664e-02 -7.02418238e-02
6.37866184e-03 1.10927224e-02 -5.46942912e-02 -2.65524238e-02
4.21768017e-02 2.57787965e-02 1.73149984e-02 -8.90427530e-02
-2.06724703e-02 -5.22891991e-03 6.91312365e-03 5.12088127e-02
-4.68316972e-02 4.63835448e-02 -2.44269054e-02 1.00091763e-01
-1.27162814e-01 3.28483544e-02 9.54644978e-02 -2.42244136e-02
-5.49561344e-03 -5.11313938e-02 -9.60426312e-03 -1.80531517e-02
6.04213662e-02 1.47664882e-02 -7.60244802e-02 -2.43510772e-02
master*+ 0 0 Git Graph
Ln 241, Col 20 Spaces: 4 UTF-8 CRLF Python Stanza venv (3.14.2)
28°C Sunny
Search
ENG IN
11:28 AM 26-09-2026


File Edit Selection View Go Run
Resume-JD-RAG-Analyzer
Problems Output Debug Console Terminal Ports Stanza
powershell
(resume-rag) PS C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer> python src/vector_store.py
-1.13757960e-01 6.45723939e-02 2.65065976e-03 3.70353982e-02
6.40230114e-03 -4.97911870e-02 1.80003028e-02 2.82854345e-02
8.87091011e-02 2.14385279e-02 3.12929675e-02 -4.01879326e-02
8.61951038e-02 -5.10971388e-03 -8.33972842e-02 6.19562082e-02
-2.48771720e-03 3.74960490e-02 -4.40172926e-02 -6.16535729e-08
-4.04837914e-02 -1.15163671e-02 2.16134917e-02 -3.28042433e-02
3.82401496e-02 1.18492106e-02 -3.99619751e-02 2.20966153e-03
1.28536380e-03 2.23805103e-03 3.56634259e-02 -1.21338330e-02
2.64358125e-04 1.95879489e-03 7.83073083e-02 -4.06366214e-03
-4.23494121e-03 2.36343760e-02 -1.07360454e-02 -1.07765989e-02
1.47037730e-01 -4.58280407e-02 9.03295353e-03 6.33321237e-03
3.72460438e-03 -5.05056605e-02 -3.19995619e-02 4.17364351e-02
-7.27977529e-02 1.62328314e-02 -3.99282798e-02 4.01186049e-02
6.11719415e-02 -4.01485562e-02 1.35698775e-02 1.07055992e-01
4.24597189e-02 -7.66800940e-02 -3.98087837e-02 6.23673610e-02
-6.41670078e-02 4.67142612e-02 -1.47534609e-02 -9.84485960e-04
1.00668035e-02 3.60974707e-02 3.03500574e-02 -9.71080139e-02
3.50388661e-02 1.50018418e-02 -1.04218768e-02 -3.81152034e-02
5.83486594e-02 8.97358581e-02 8.47530141e-02 1.86756179e-02
-3.07655474e-03 -3.21688093e-02 -2.89569385e-02 7.59645328e-02
4.61857021e-02 2.62104888e-02 -3.88625599e-02 9.30876355e-04]
First vector length: 384
Testing question embedding:
Question: What skills are required for this job?
Question embedding shape: (384,)
==
EMBEDDING TEST COMPLETE
==
(resume-rag) PS C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer>
master*+ 0 0 Git Graph
Ln 241, Col 20 Spaces: 4 UTF-8 CRLF Python Stanza venv (3.14.2)
28°C Sunny
Search
ENG IN
11:28 AM 26-09-2026


File Edit Selection View Go Run
Resume-JD-RAG-Analyzer
.gitignore U
.env
file_loader.py U
embeddings.py U
vector_store.py U
retriever.py U
chunking.py U
src > retriever.py > ...
Problems Output Debug Console Terminal Ports Stanza
powershell
(resume-rag) PS C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer> python src/vector_store.py
Total chunks: 22
Creating embeddings...
Batches: 100%| 1/1 [00:00<00:00, 2.61it/s]
Embedding shape: (22, 384)
Embedding dimension: 384
Vectors added to FAISS: 22
FAISS index saved:
C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer\vector_store\index.faiss
Metadata saved:
C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer\vector_store\metadata.pkl
==
VECTOR STORE CREATED SUCCESSFULLY
==
FAISS vectors : 22
Metadata items: 22
index.faiss size: 33837 bytes
metadata.pkl size: 10072 bytes
(resume-rag) PS C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer> Get-ChildItem .\vector_store | Select-Object Name, Length
Name Length
---- ------
index.faiss 33837
metadata.pkl 10072
master*+ 0 0 Git Graph
Ln 449, Col 20 Spaces: 4 UTF-8 CRLF Python Stanza venv (3.14.2)
31°C Sunny
Search
ENG IN
11:47 AM 26-09-2026

5. What is actually inside these files?

You don't manually open them.

`index.faiss`

Contains your numerical embeddings:

```
Plain text
Resume chunk 1 → 384 numbers
Resume chunk 2 → 384 numbers
JD chunk 1     → 384 numbers
JD chunk 2     → 384 numbers
...
```

For example:

```
Plain text
[0.012, -0.031, 0.224, ...]
```

FAISS uses these vectors for simi.. ..y search.

`metadata.pkl`

Contains information corresponding to each vector:

Python

Run

```
[
    {
        "text": "Python, Flask, Pandas, NumPy...",
        "source": "Resume.pdf",
        "page": 1,
        "document_type": "resume"
    },

    {
        "text": "Required skills include Python...",
        "source": "Job_Description.pdf",
        "page": 1,
        "document_type": "job_description"
    }
]
```

The important relationship is:

Plain text

```
FAISS vector #0  ↔ metadata #0
FAISS vector #1  ↔ metadata #1
FAISS vector #2  ↔ metadata #2
```

4. Check the files

Run:

```powershell
Get-ChildItem .\vector_store | Select-Object Name, Length
```

You should **NOT** see:

```
index.faiss       0
metadata.pkl      0
```

You should see something like:

```
Name            Length
----            ------
index.faiss      18452
metadata.pkl     9231
```

The exact sizes will be different.

STEP 8 — Build retriever

Create:

`Backend/src/retriever.py`

Purpose:

```
User Question
      ↓
Question Embedding
      ↓
FAISS
      ↓
Top relevant chunks
```

But this project needs something extra.

You should be able to retrieve specifically from:

```
Resume
```

or:

```
Job Description
```

or:

```
Both
```

For example:

```
Question:
What Python skills does the candidate have?

        ↓

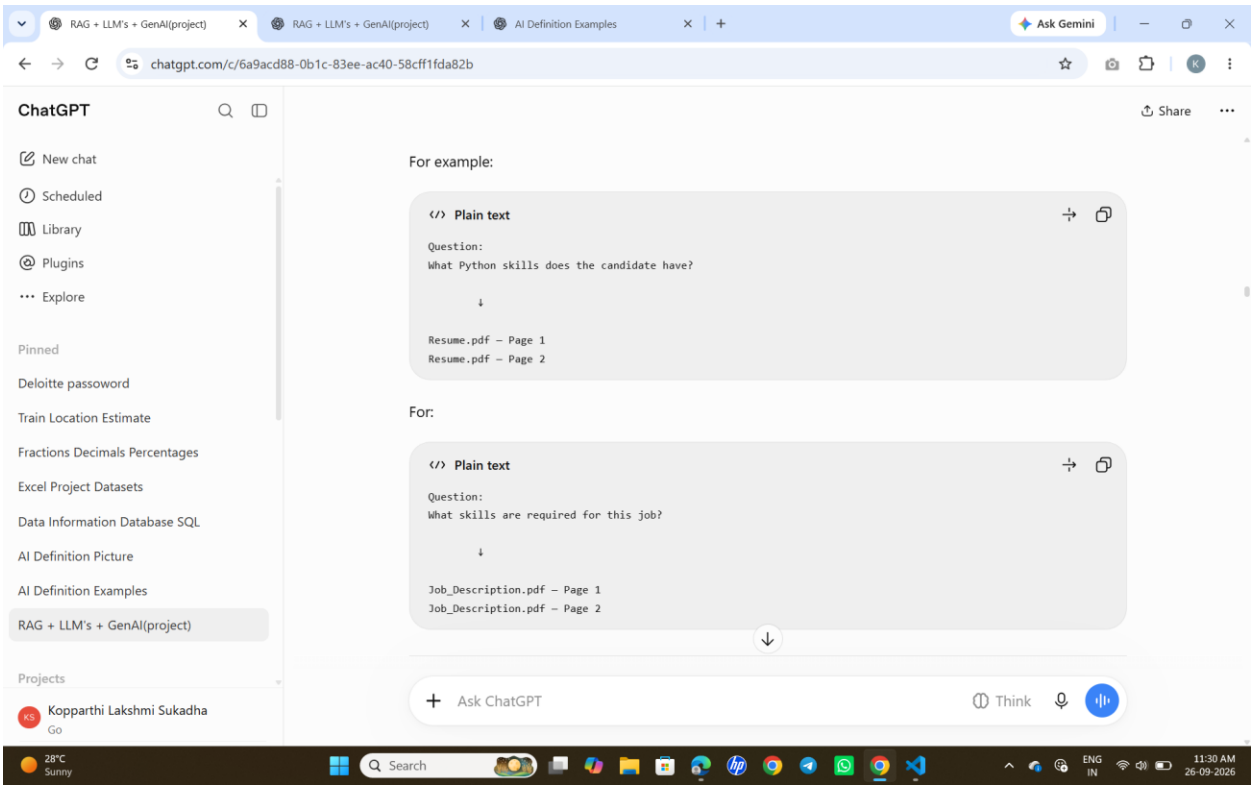
Resume.pdf — Page 1
Resume.pdf — Page 2
```

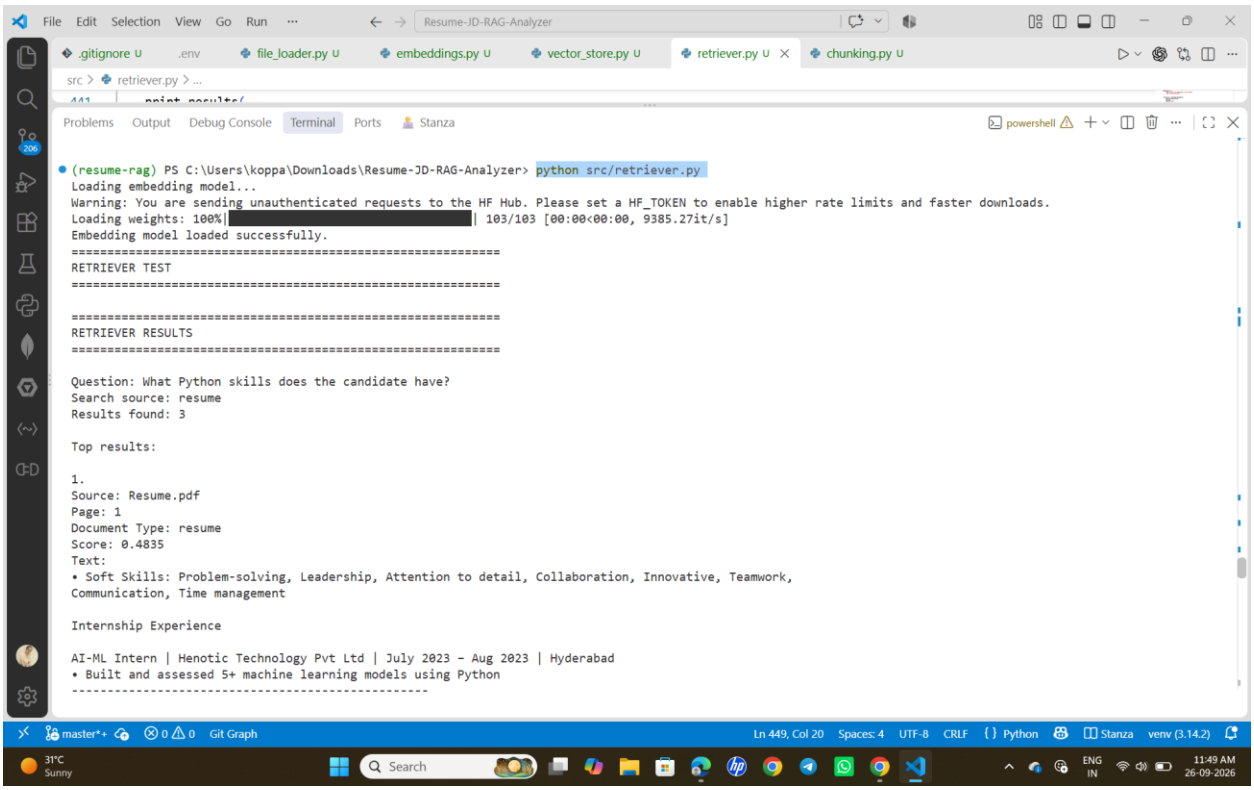
For:

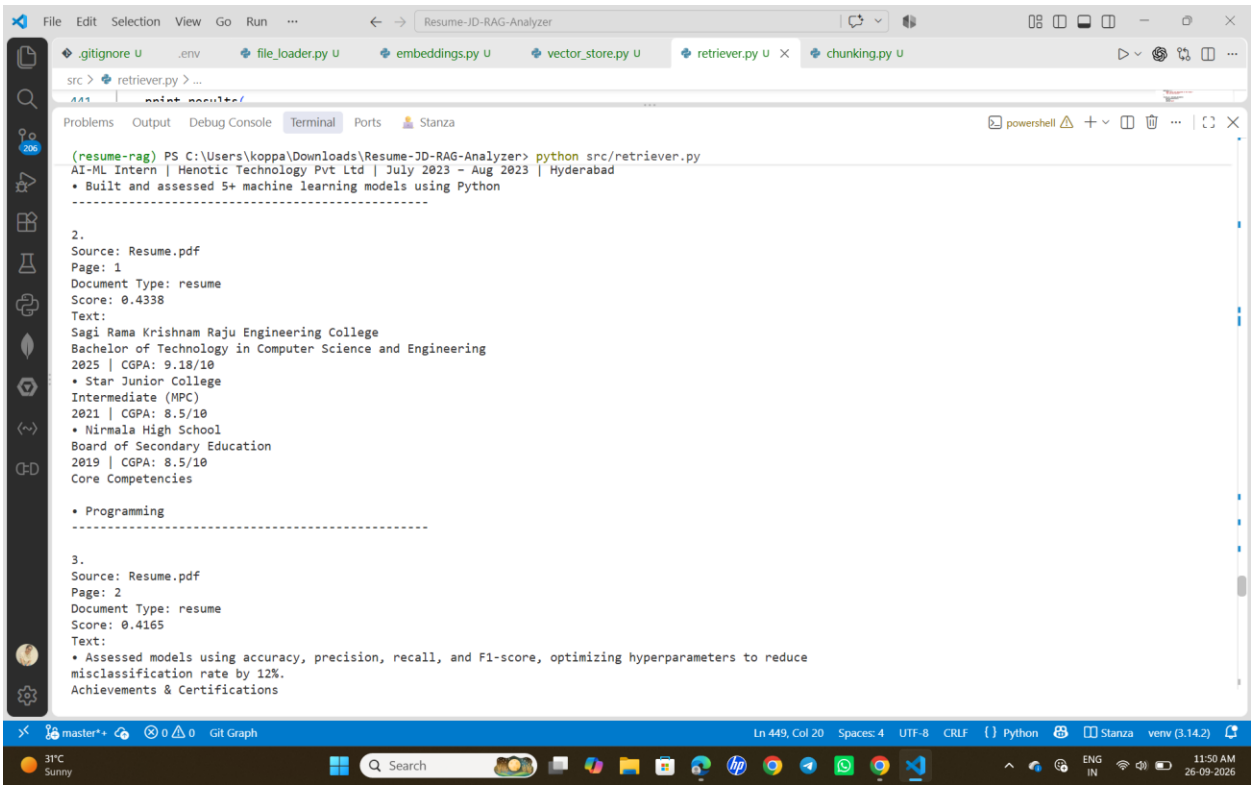
```
Question:
What skills are required for this job?

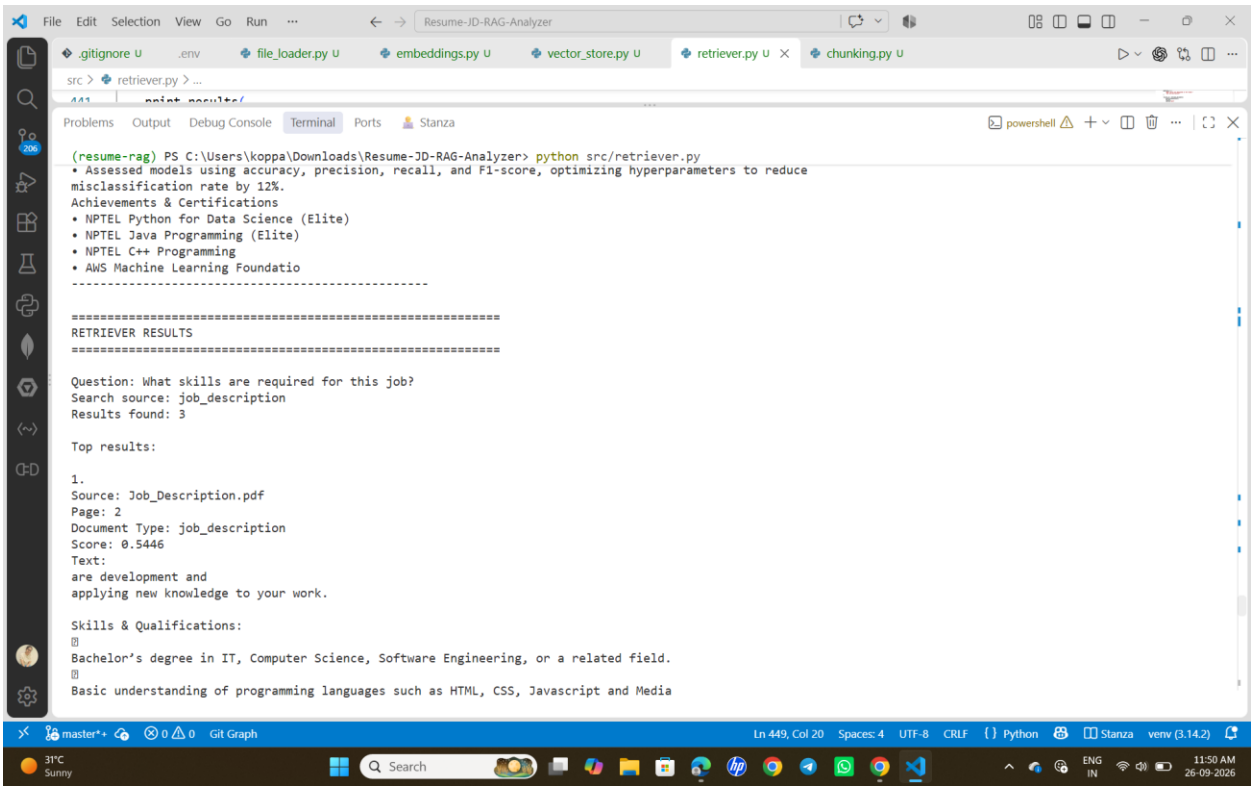
        ↓

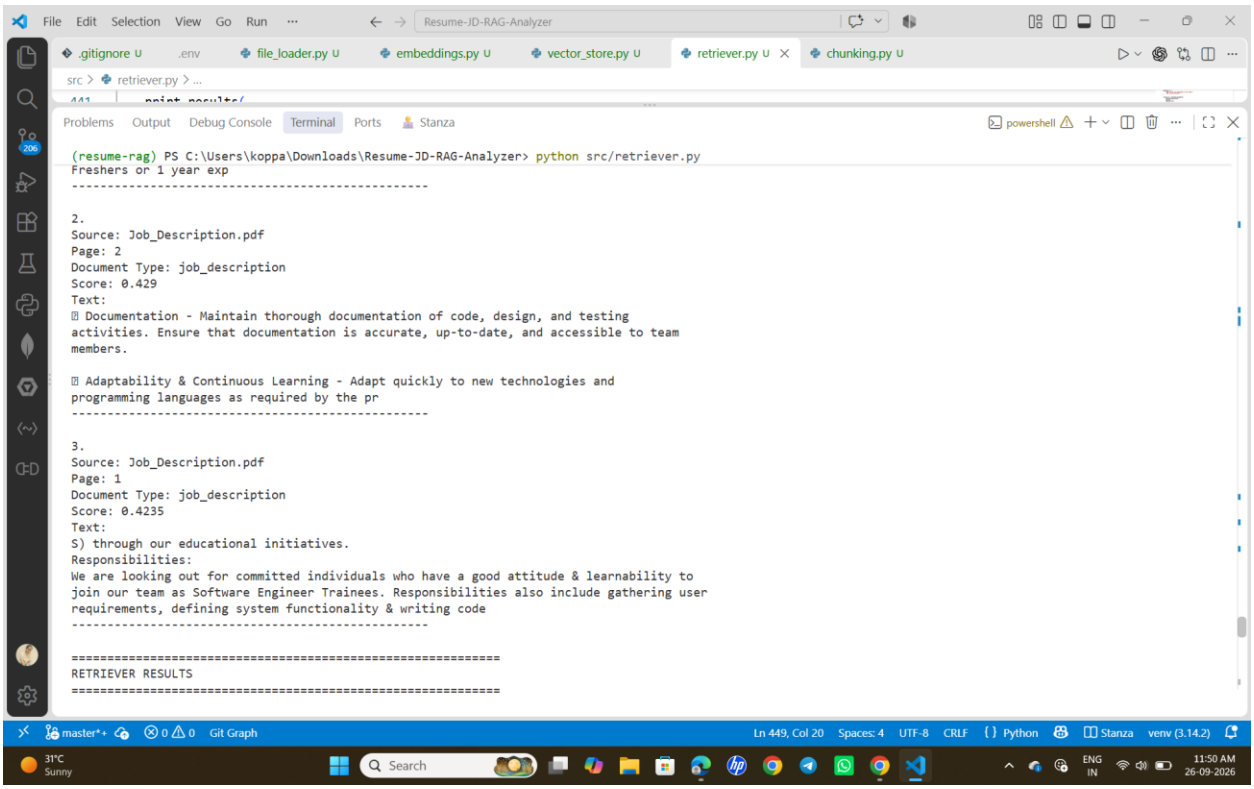
Job_Description.pdf — Page 1
Job_Description.pdf — Page 2
```


RAG + LLM's + GenAI(project)
RAG + LLM's + GenAI(project)
AI Definition Examples
Ask Gemini
chatgpt.com/c/6a9acd88-0b1c-83ee-ac40-58cff1fda82b
ChatGPT
Share
New chat
Scheduled
Library
Plugins
Explore
Pinned
Deloitte passoword
Train Location Estimate
Fractions Decimals Percentages
Excel Project Datasets
Data Information Database SQL
AI Definition Picture
AI Definition Examples
RAG + LLM's + GenAI(project)
Projects
Kopparthi Lakshmi Sukadha
Go
For example:
Plain text
Question:
What Python skills does the candidate have?
↓
Resume.pdf — Page 1
Resume.pdf — Page 2
For:
Plain text
Question:
What skills are required for this job?
↓
Job_Description.pdf — Page 1
Job_Description.pdf — Page 2
Ask ChatGPT
Think
28°C
Sunny
Search
ENG
IN
11:30 AM
26-09-2026


File Edit Selection View Go Run
Resume-JD-RAG-Analyzer
.gitignore U
.env
file_loader.py U
embeddings.py U
vector_store.py U
retriever.py U
chunking.py U
src > retriever.py > ...
Problems
Output
Debug Console
Terminal
Ports
Stanza
powershell
(resume-rag) PS C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer> python src/retriever.py
Loading embedding model...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%| | 103/103 [00:00<00:00, 9385.27it/s]
Embedding model loaded successfully.
==
RETRIEVER TEST
==
==
RETRIEVER RESULTS
==
Question: What Python skills does the candidate have?
Search source: resume
Results found: 3
Top results:
1.
Source: Resume.pdf
Page: 1
Document Type: resume
Score: 0.4835
Text:
• Soft Skills: Problem-solving, Leadership, Attention to detail, Collaboration, Innovative, Teamwork,
Communication, Time management
Internship Experience
AI-ML Intern | Henotic Technology Pvt Ltd | July 2023 – Aug 2023 | Hyderabad
• Built and assessed 5+ machine learning models using Python
master*+
Git Graph
Ln 449, Col 20
Spaces: 4
UTF-8
CRLF
Python
Stanza
venv (3.14.2)
31°C
Sunny
Search
ENG
IN
11:49 AM
26-09-2026


File Edit Selection View Go Run
Resume-JD-RAG-Analyzer
.gitignore
.env
file_loader.py
embeddings.py
vector_store.py
retriever.py
chunking.py
src > retriever.py > ...
Problems Output Debug Console Terminal Ports Stanza
powershell
(resume-rag) PS C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer> python src/retriever.py
AI-ML Intern | Henotic Technology Pvt Ltd | July 2023 - Aug 2023 | Hyderabad
• Built and assessed 5+ machine learning models using Python
2.
Source: Resume.pdf
Page: 1
Document Type: resume
Score: 0.4338
Text:
Sagi Rama Krishnam Raju Engineering College
Bachelor of Technology in Computer Science and Engineering
2025 | CGPA: 9.18/10
• Star Junior College
Intermediate (MPC)
2021 | CGPA: 8.5/10
• Nirmala High School
Board of Secondary Education
2019 | CGPA: 8.5/10
Core Competencies
• Programming
3.
Source: Resume.pdf
Page: 2
Document Type: resume
Score: 0.4165
Text:
• Assessed models using accuracy, precision, recall, and F1-score, optimizing hyperparameters to reduce
misclassification rate by 12%.
Achievements & Certifications
master*+
Git Graph
Ln 449, Col 20
Spaces: 4
UTF-8
CRLF
Python
Stanza
venv (3.14.2)
31°C
Sunny
Search
ENG
IN
11:50 AM
26-09-2026

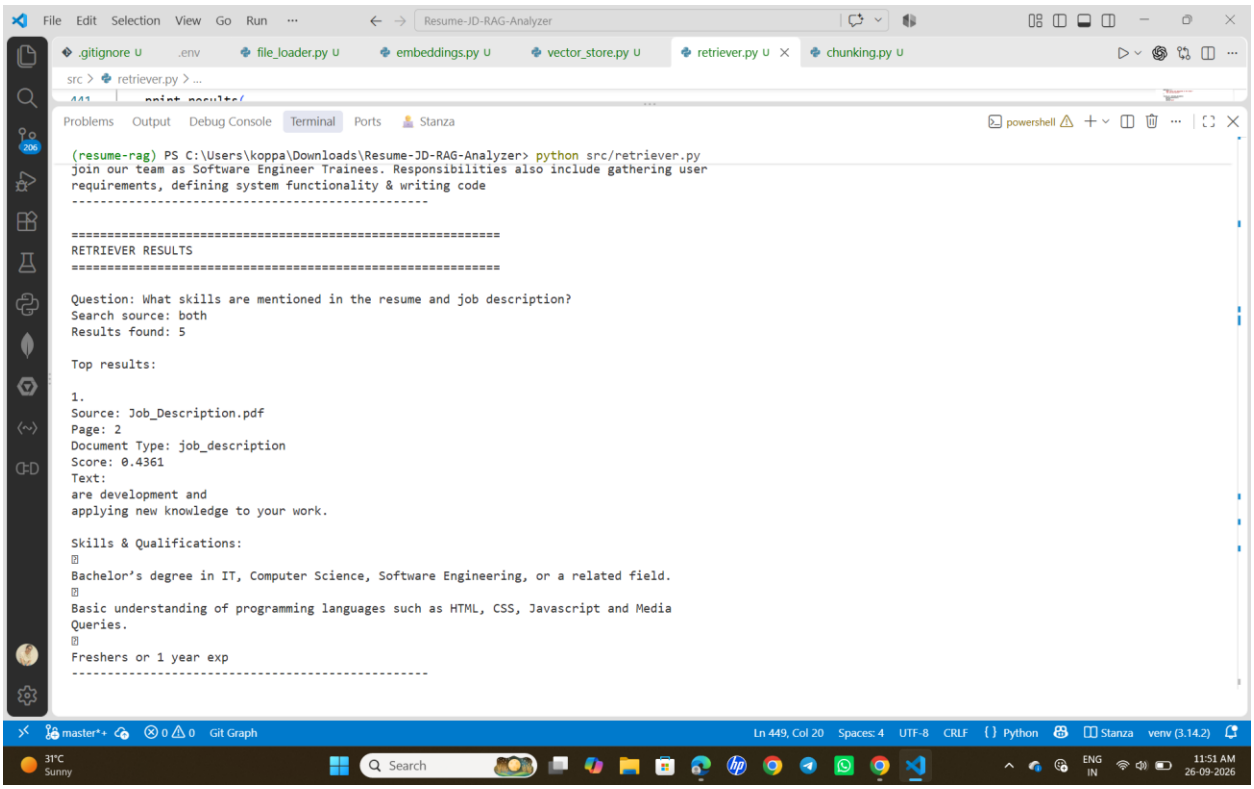

File Edit Selection View Go Run
Resume-JD-RAG-Analyzer
.gitignore
.env
file_loader.py
embeddings.py
vector_store.py
retriever.py
chunking.py
src > retriever.py > ...
Problems Output Debug Console Terminal Ports Stanza
powershell
(resume-rag) PS C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer> python src/retriever.py
• Assessed models using accuracy, precision, recall, and F1-score, optimizing hyperparameters to reduce
misclassification rate by 12%.
Achievements & Certifications
• NPTEL Python for Data Science (Elite)
• NPTEL Java Programming (Elite)
• NPTEL C++ Programming
• AWS Machine Learning Foundatio
RETRIEVER RESULTS
Question: What skills are required for this job?
Search source: job_description
Results found: 3
Top results:
1.
Source: Job_Description.pdf
Page: 2
Document Type: job_description
Score: 0.5446
Text:
are development and
applying new knowledge to your work.
Skills & Qualifications:
Bachelor's degree in IT, Computer Science, Software Engineering, or a related field.
Basic understanding of programming languages such as HTML, CSS, Javascript and Media
master*+
Git Graph
Ln 449, Col 20
Spaces: 4
UTF-8
CRLF
Python
Stanza
venv (3.14.2)
31°C
Sunny
Search
ENG
IN
11:50 AM
26-09-2026


File Edit Selection View Go Run
Resume-JD-RAG-Analyzer
.gitignore U
.env
file_loader.py U
embeddings.py U
vector_store.py U
retriever.py U
chunking.py U
src > retriever.py > ...
Problems Output Debug Console Terminal Ports Stanza
powershell
(resume-rag) PS C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer> python src/retriever.py
Freshers or 1 year exp

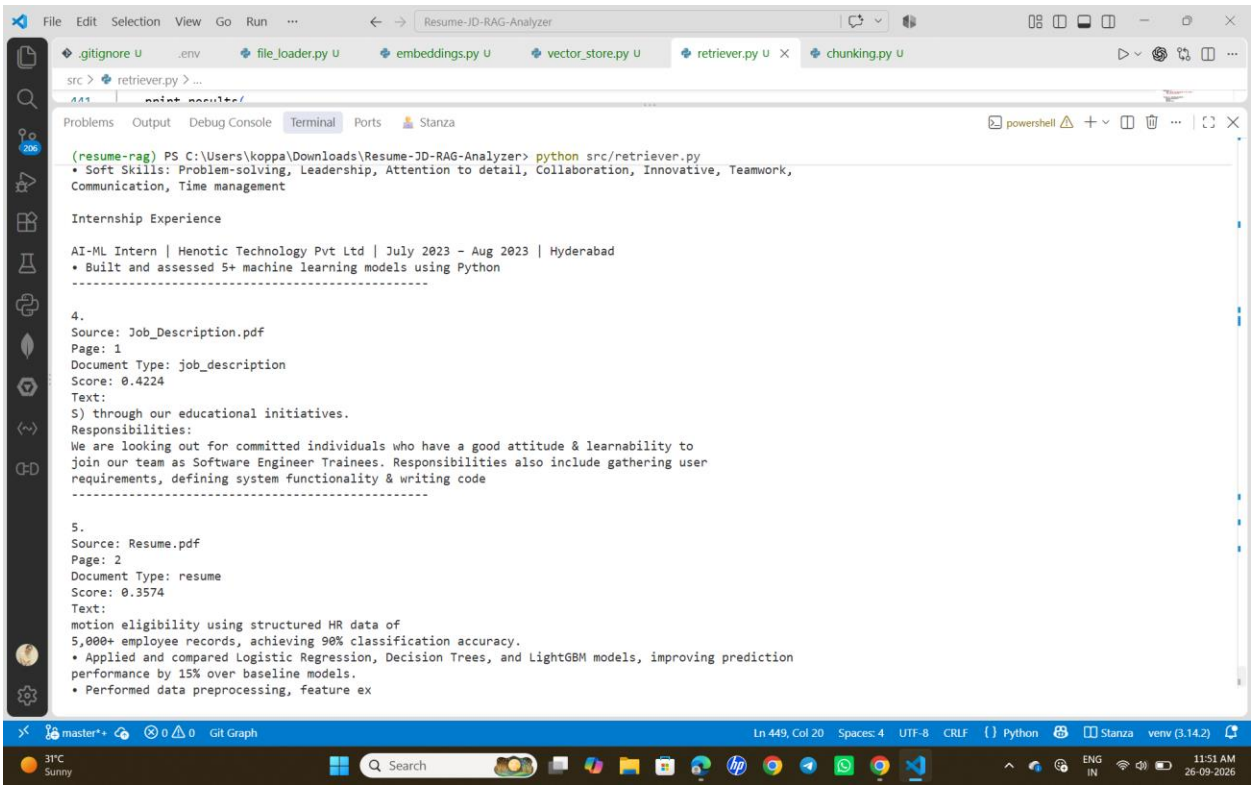
2.
Source: Job_Description.pdf
Page: 2
Document Type: job_description
Score: 0.429
Text:
Documentation - Maintain thorough documentation of code, design, and testing
activities. Ensure that documentation is accurate, up-to-date, and accessible to team
members.
Adaptability & Continuous Learning - Adapt quickly to new technologies and
programming languages as required by the pr

3.
Source: Job_Description.pdf
Page: 1
Document Type: job_description
Score: 0.4235
Text:
S) through our educational initiatives.
Responsibilities:
We are looking out for committed individuals who have a good attitude & learnability to
join our team as Software Engineer Trainees. Responsibilities also include gathering user
requirements, defining system functionality & writing code

==
RETRIEVER RESULTS
==
master*+ 0 0 0 Git Graph
Ln 449, Col 20 Spaces: 4 UTF-8 CRLF Python Stanza venv (3.14.2)
31°C Sunny
Search
ENG IN
11:50 AM 26-09-2026


File Edit Selection View Go Run
Resume-JD-RAG-Analyzer
.gitignore U
.env
file_loader.py U
embeddings.py U
vector_store.py U
retriever.py U
chunking.py U
src > retriever.py > ...
Problems Output Debug Console Terminal Ports Stanza
powershell
(resume-rag) PS C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer> python src/retriever.py
join our team as Software Engineer Trainees. Responsibilities also include gathering user
requirements, defining system functionality & writing code

==
RETRIEVER RESULTS
==
Question: What skills are mentioned in the resume and job description?
Search source: both
Results found: 5
Top results:
1.
Source: Job_Description.pdf
Page: 2
Document Type: job_description
Score: 0.4361
Text:
are development and
applying new knowledge to your work.
Skills & Qualifications:
Bachelor's degree in IT, Computer Science, Software Engineering, or a related field.
Basic understanding of programming languages such as HTML, CSS, Javascript and Media
Queries.
Freshers or 1 year exp
---
master*+ 0 0 0 Git Graph
Ln 449, Col 20 Spaces: 4 UTF-8 CRLF Python Stanza venv (3.14.2)
31°C Sunny
Search
ENG IN
11:51 AM 26-09-2026



STEP 9 — Build Gemini LLM

Create:

```
Backend/src/llm.py
```

Gemini receives:

```
Question
+
Retrieved Resume Context
+
Retrieved JD Context
```

Example:

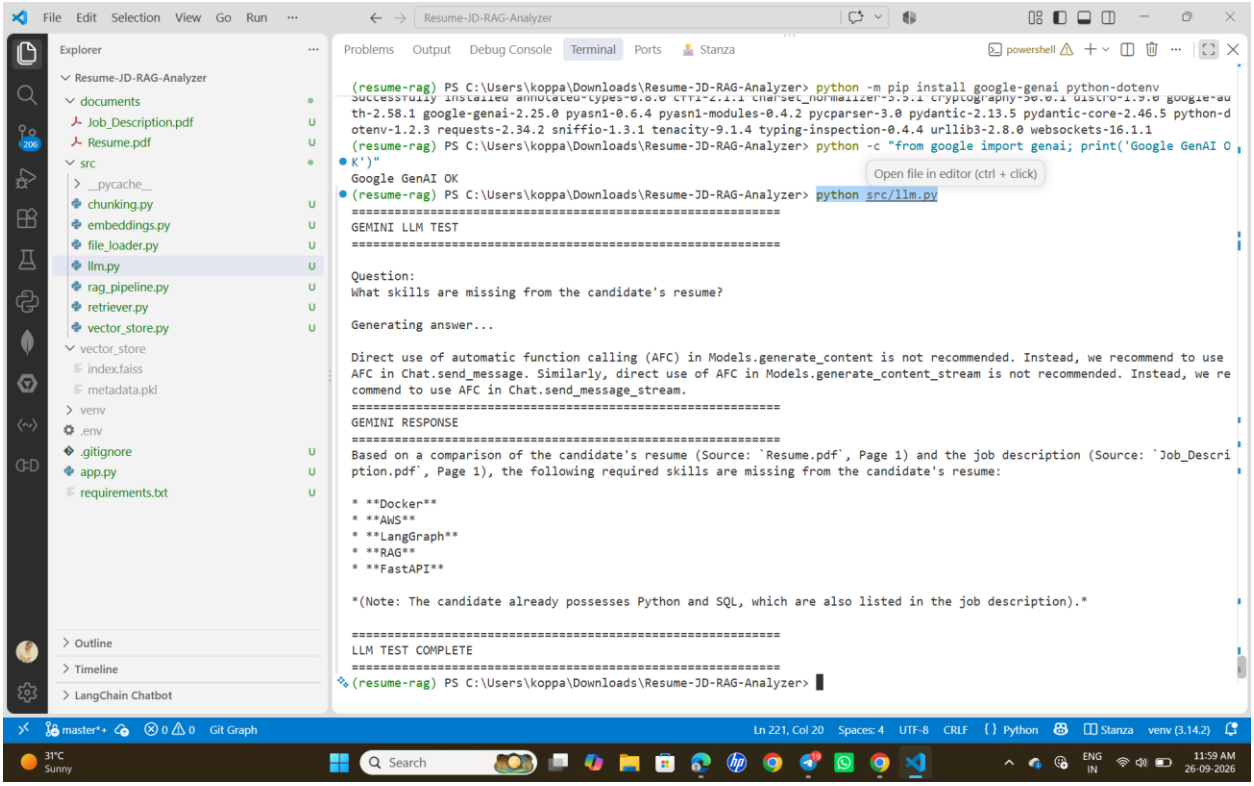
```
Question:
What skills are missing?

Resume Context:
```

```
Python
Flask
React
SQL
Machine Learning

JD Context:
Python
SQL
Docker
AWS
LangGraph
RAG
FastAPI
```

Gemini then produces the analysis.



STEP 10 — Design the RAG prompt

This is one of the **most important parts of your project**.

Give Gemini instructions such as:

```
You are a Resume and Job Description Analyzer.

Analyze the candidate's resume against the provided job description.

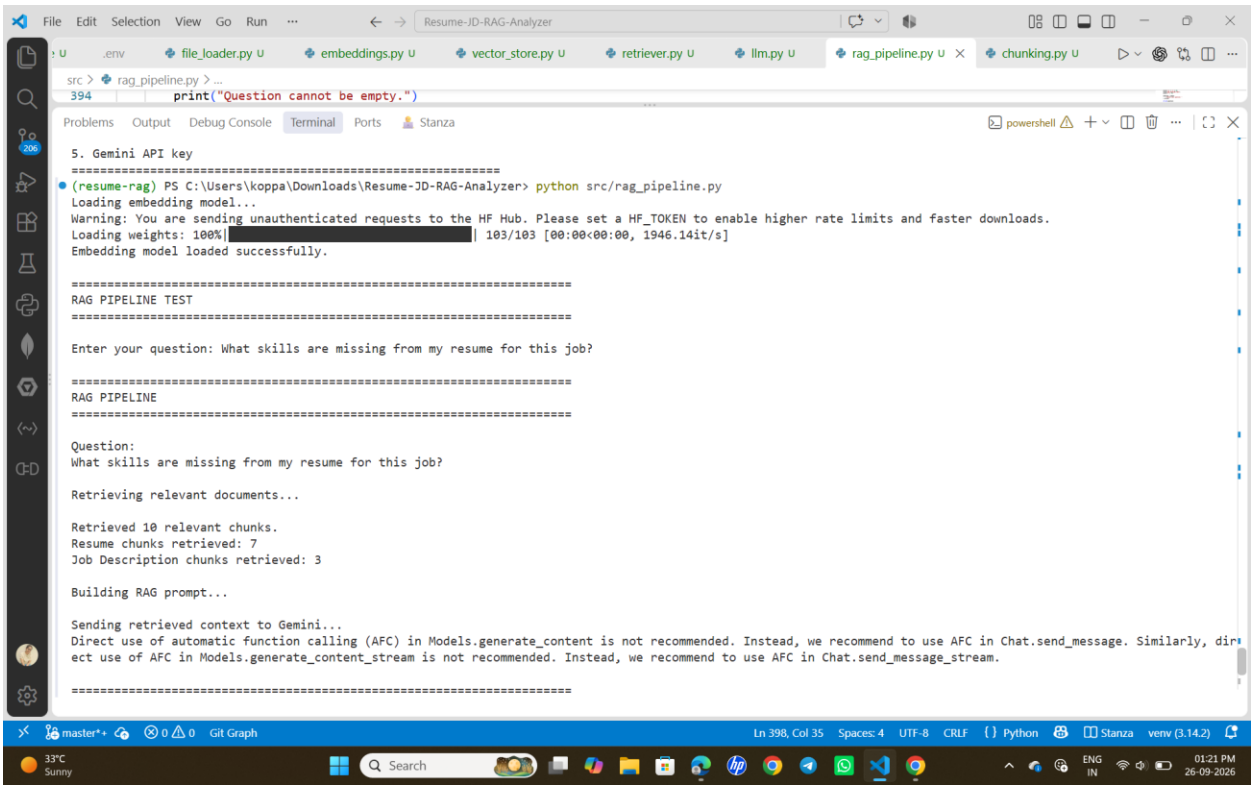
Use the retrieved resume and job description context as the primary
source.

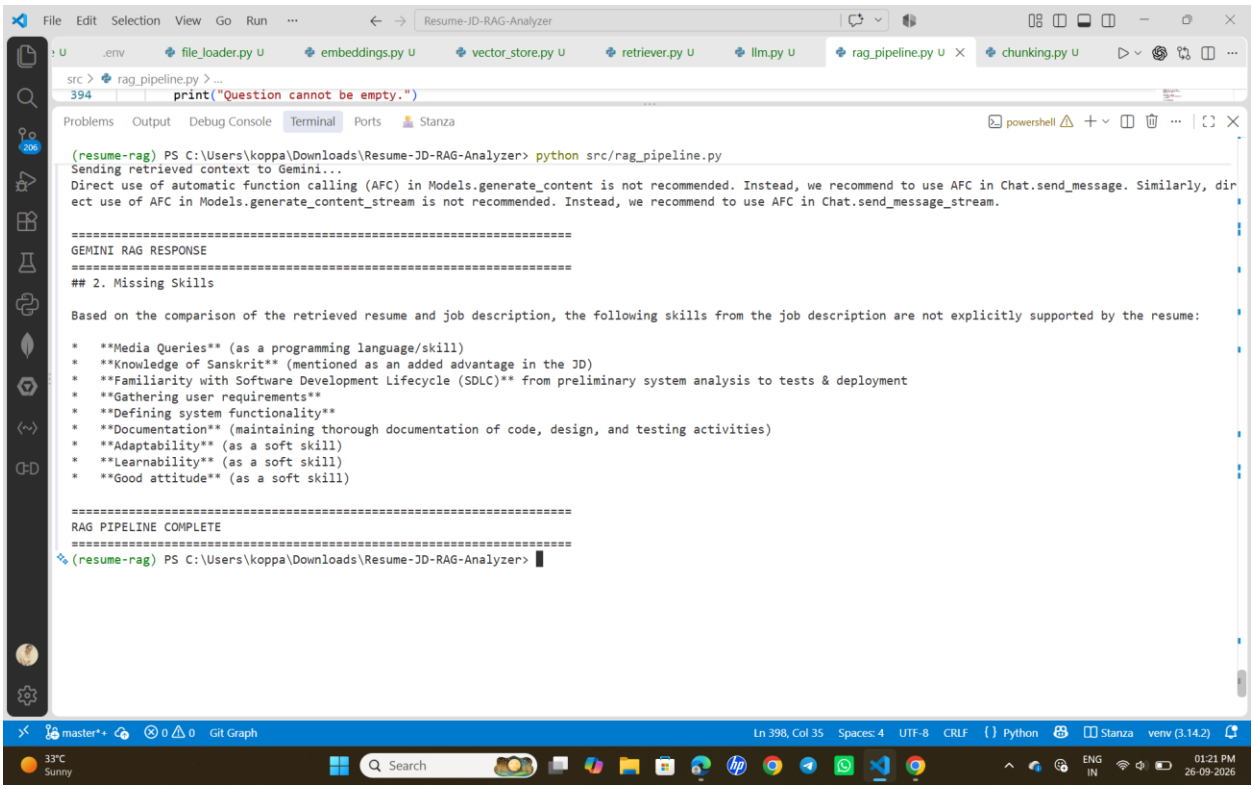
Do not invent skills, projects, certifications, or experience that are
not supported by the provided documents.

Identify:
1. Matching skills
2. Missing skills
3. Relevant projects
4. Resume improvement suggestions
5. Interview questions
6. Technical questions
7. Relevant resume sections

Clearly distinguish information found in the resume from requirements
found in the job description.
```

This makes your project a proper **RAG application**, rather than simply sending both PDFs directly to an LLM.


File Edit Selection View Go Run
Resume-JD-RAG-Analyzer
.env
file_loader.py U
embeddings.py U
vector_store.py U
retriever.py U
llm.py U
rag_pipeline.py U
chunking.py U
src > rag_pipeline.py > ...
394 print("Question cannot be empty.")
Problems Output Debug Console Terminal Ports Stanza
powershell
5. Gemini API key
(resume-rag) PS C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer> python src/rag_pipeline.py
Loading embedding model...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%| 103/103 [00:00<00:00, 1946.14it/s]
Embedding model loaded successfully.
RAG PIPELINE TEST
Enter your question: What skills are missing from my resume for this job?
RAG PIPELINE
Question:
What skills are missing from my resume for this job?
Retrieving relevant documents...
Retrieved 10 relevant chunks.
Resume chunks retrieved: 7
Job Description chunks retrieved: 3
Building RAG prompt...
Sending retrieved context to Gemini...
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, dir
ect use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.
master*+ 0 0 Git Graph
Ln 398, Col 35 Spaces: 4 UTF-8 CRLF Python Stanza venv (3.14.2)
33°C Sunny
Search
ENG IN
01:21 PM 26-09-2026


File Edit Selection View Go Run
Resume-JD-RAG-Analyzer
.env
file_loader.py U
embeddings.py U
vector_store.py U
retriever.py U
llm.py U
rag_pipeline.py U
chunking.py U
src > rag_pipeline.py > ...
394 print("Question cannot be empty.")
Problems Output Debug Console Terminal Ports Stanza
powershell
(resume-rag) PS C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer> python src/rag_pipeline.py
Sending retrieved context to Gemini...
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, dir
ect use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.
GEMINI RAG RESPONSE
2. Missing Skills
Based on the comparison of the retrieved resume and job description, the following skills from the job description are not explicitly supported by the resume:
* **Media Queries** (as a programming language/skill)
* **Knowledge of Sanskrit** (mentioned as an added advantage in the JD)
* **Familiarity with Software Development Lifecycle (SDLC)** from preliminary system analysis to tests & deployment
* **Gathering user requirements**
* **Defining system functionality**
* **Documentation** (maintaining thorough documentation of code, design, and testing activities)
* **Adaptability** (as a soft skill)
* **Learnability** (as a soft skill)
* **Good attitude** (as a soft skill)
RAG PIPELINE COMPLETE
(resume-rag) PS C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer>
master*+ 0 0 Git Graph
Ln 398, Col 35 Spaces: 4 UTF-8 CRLF Python Stanza venv (3.14.2)
33°C Sunny
Search
ENG IN
01:21 PM 26-09-2026

STEP 11 — Build the analysis functions

Create:

`Backend/src/analyzer.py`

You can create separate functions for:

1. Skill Matching

```
Resume Skills
        +
JD Required Skills
        ↓
Matching Skills
```

Example:

```
Matching Skills:
✓ Python
✓ SQL
✓ Machine Learning
✓ Flask
✓ RAG
```

2. Missing Skills

```
JD Requirements
        -
Resume Skills
        ↓
Potentially Missing Skills
```

Example:

```
Potential gaps:
• Docker
• Kubernetes
```

- `LangGraph`
- `Azure`

Use wording like **"not found in the resume"** rather than assuming the candidate definitely lacks the skill.

3. Relevant Projects

The system should identify projects from the resume that relate to the JD.

Example:

```
JD Requirement:
Generative AI / RAG

Relevant Resume Project:
AI Study Assistant

Why relevant:
The project uses embeddings, FAISS, RAG and Gemini.
```

4. Resume Improvement Suggestions

Example:

```
Current:
Developed an AI chatbot.

Suggestion:
Add the technologies, architecture and measurable result
if those details are supported by the candidate's actual experience.
```

Don't allow Gemini to invent metrics.

5. Interview Questions

Generate questions based on the JD:

```
1. Explain RAG architecture.
2. What is an embedding?
3. Why would you use a vector database?
4. How does FAISS perform similarity search?
5. How would you reduce hallucinations in an RAG system?
```

6. Technical Questions Based on Resume

Example:

If the resume contains:

```
Python
Flask
TensorFlow
NLP
FAISS
RAG
```

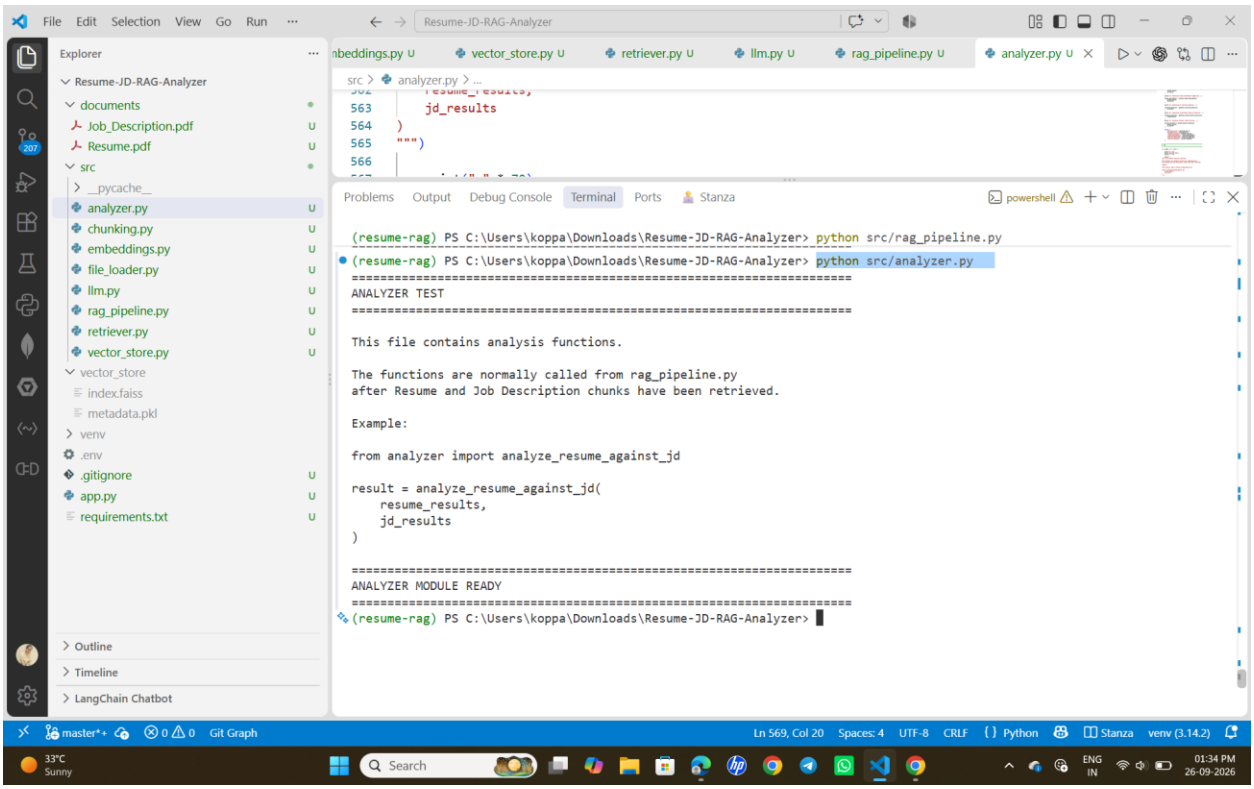
generate questions around those technologies.

7. Section Relevance

Example:

```
Project: AI Study Assistant

Relevant because:
The JD requires RAG, embeddings and LLM-based applications,
and the project demonstrates those technologies.
```



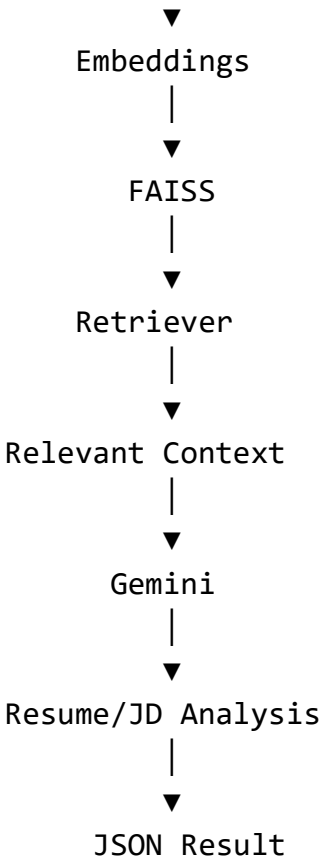
STEP 12 — Build the complete RAG pipeline

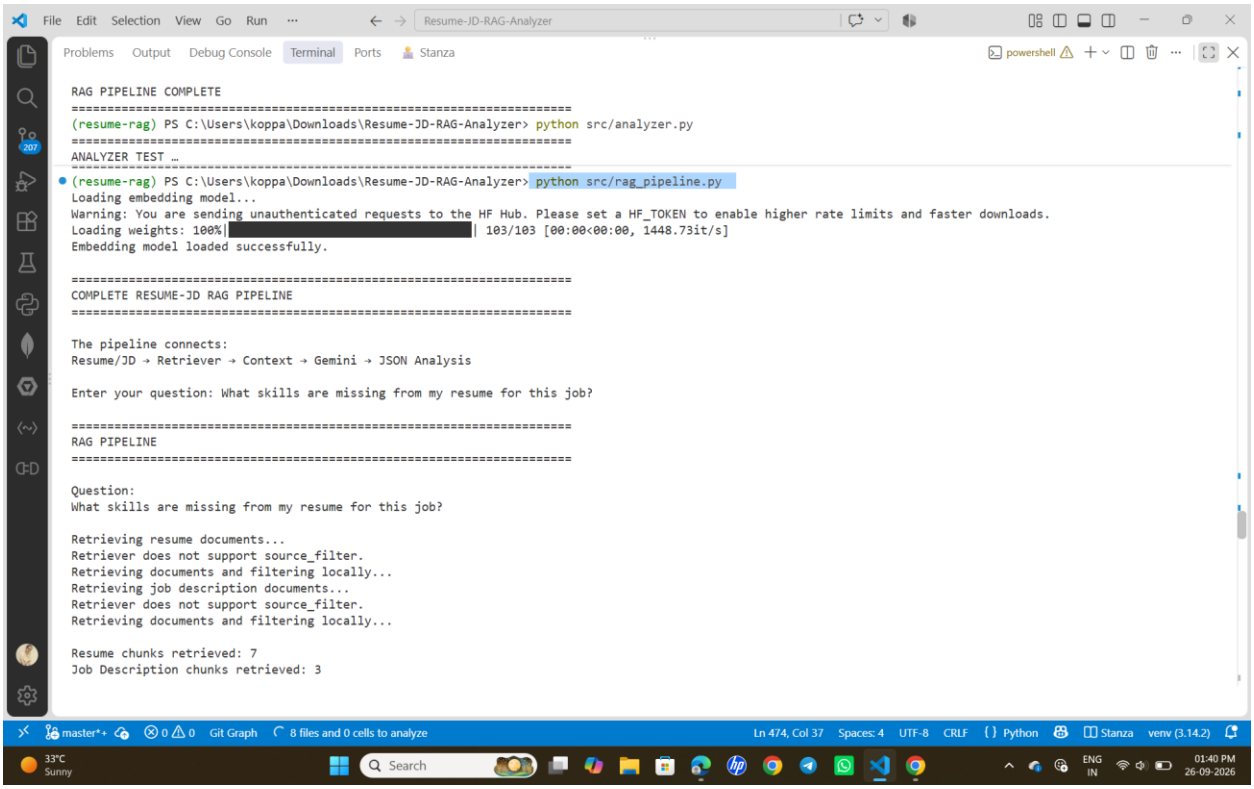
Create:

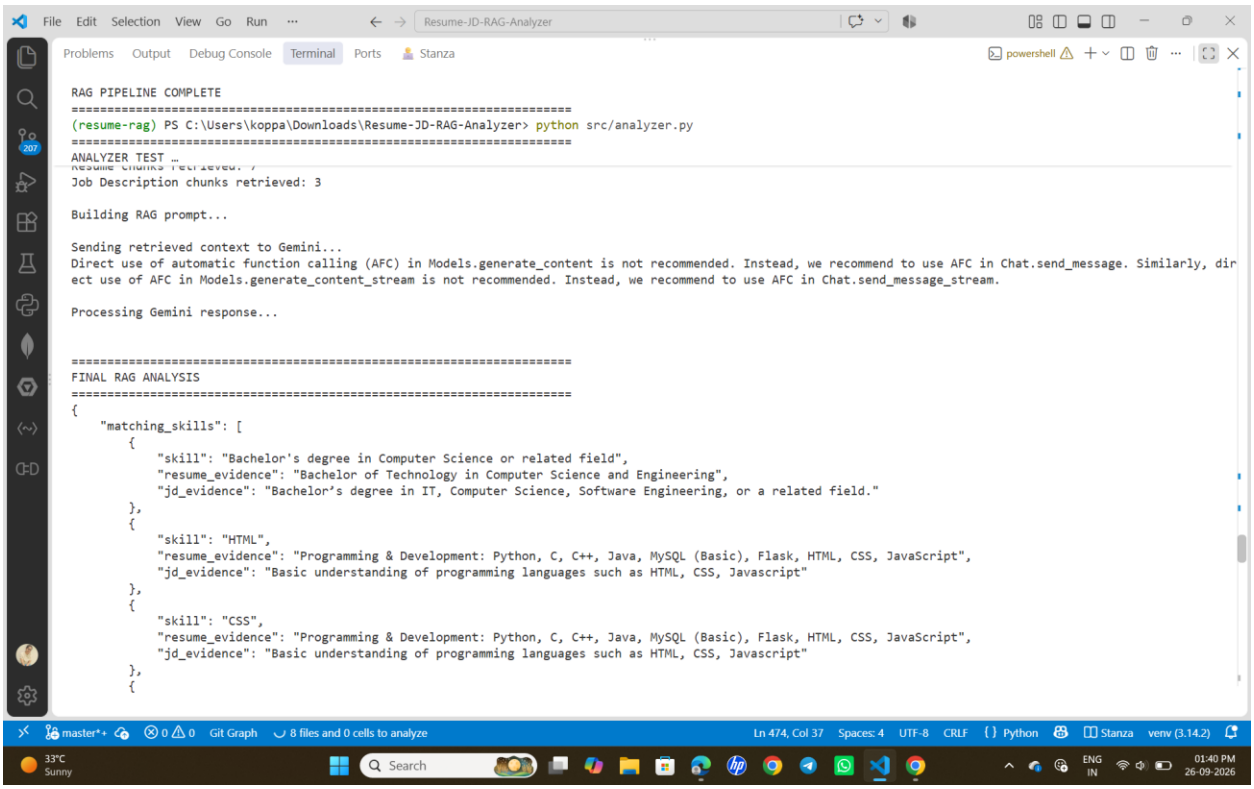
`Backend/src/rag_pipeline.py`

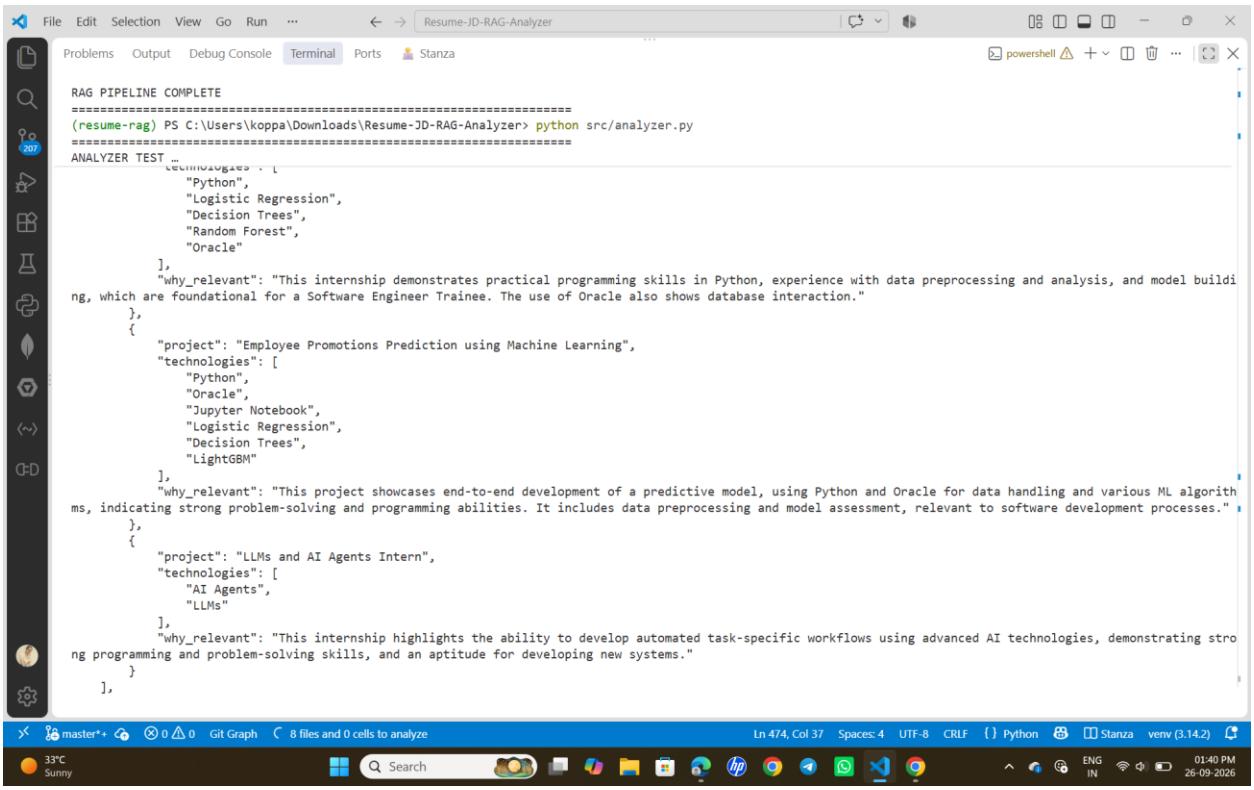
Final flow:

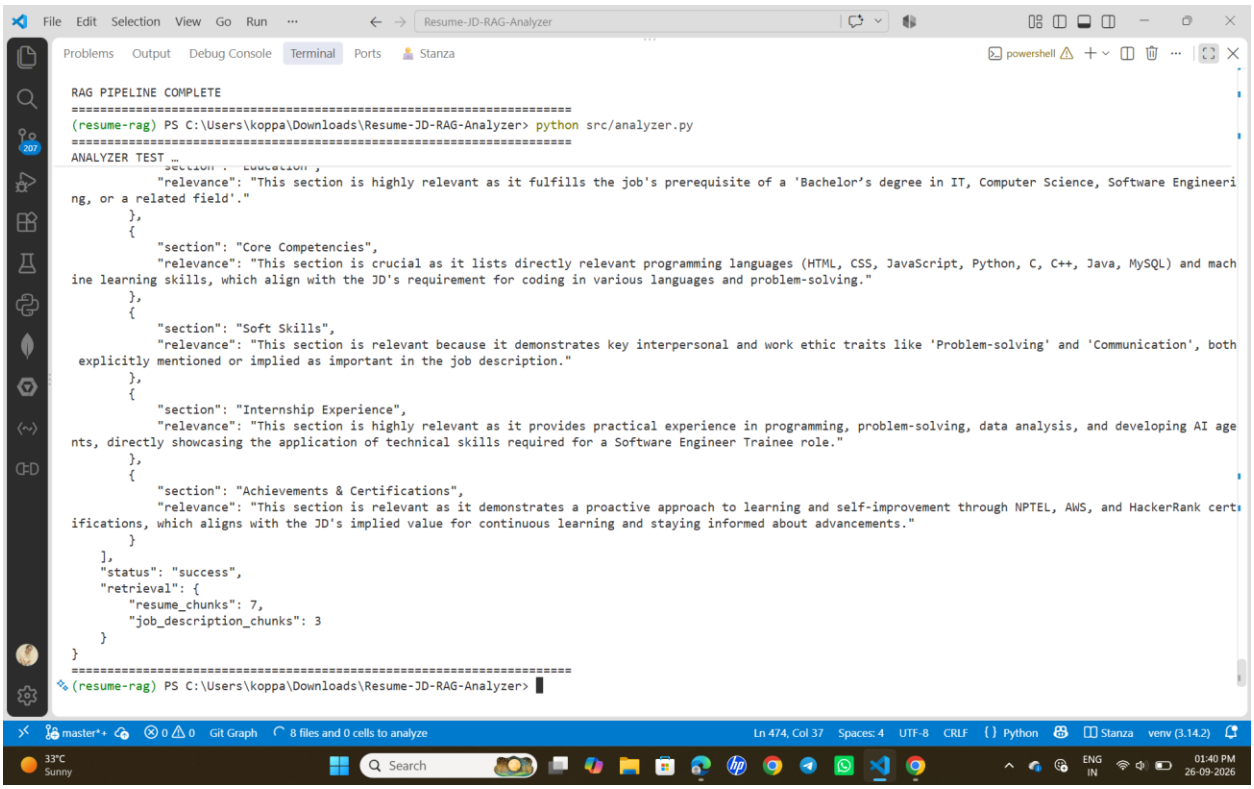
```
                USER
                  |
                  ▼
Resume + Job Description
                  |
                  ▼
        Document Parser
                  |
                  ▼
              Chunking
                  |
```


Embeddings
FAISS
Retriever
Relevant Context
Gemini
Resume/JD Analysis
JSON Result


RAG PIPELINE COMPLETE
(resume-rag) PS C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer> python src/analyzer.py
ANALYZER TEST …
(resume-rag) PS C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer> python src/rag_pipeline.py
Loading embedding model...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%| 103/103 [00:00<00:00, 1448.73it/s]
Embedding model loaded successfully.
COMPLETE RESUME-JD RAG PIPELINE
The pipeline connects:
Resume/JD → Retriever → Context → Gemini → JSON Analysis
Enter your question: What skills are missing from my resume for this job?
RAG PIPELINE
Question:
What skills are missing from my resume for this job?
Retrieving resume documents...
Retriever does not support source_filter.
Retrieving documents and filtering locally...
Retrieving job description documents...
Retriever does not support source_filter.
Retrieving documents and filtering locally...
Resume chunks retrieved: 7
Job Description chunks retrieved: 3


File Edit Selection View Go Run
Resume-JD-RAG-Analyzer
Problems Output Debug Console Terminal Ports Stanza
powershell
RAG PIPELINE COMPLETE
(resume-rag) PS C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer> python src/analyzer.py
ANALYZER TEST …
Job Description chunks retrieved: 3
Building RAG prompt...
Sending retrieved context to Gemini...
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, dir
ect use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.
Processing Gemini response...
FINAL RAG ANALYSIS
{
"matching_skills": [
{
"skill": "Bachelor's degree in Computer Science or related field",
"resume_evidence": "Bachelor of Technology in Computer Science and Engineering",
"jd_evidence": "Bachelor's degree in IT, Computer Science, Software Engineering, or a related field."
},
{
"skill": "HTML",
"resume_evidence": "Programming & Development: Python, C, C++, Java, MySQL (Basic), Flask, HTML, CSS, JavaScript",
"jd_evidence": "Basic understanding of programming languages such as HTML, CSS, Javascript"
},
{
"skill": "CSS",
"resume_evidence": "Programming & Development: Python, C, C++, Java, MySQL (Basic), Flask, HTML, CSS, JavaScript",
"jd_evidence": "Basic understanding of programming languages such as HTML, CSS, Javascript"
},
{
master*+ 0 0 Git Graph 8 files and 0 cells to analyze
Ln 474, Col 37 Spaces: 4 UTF-8 CRLF Python Stanza venv (3.14.2)
33°C Sunny
Search
ENG IN
01:40 PM 26-09-2026


File Edit Selection View Go Run
Resume-JD-RAG-Analyzer
Problems Output Debug Console Terminal Ports Stanza
powershell
RAG PIPELINE COMPLETE
(resume-rag) PS C:\Users\koppa\Downloads\Resume-JD-RAG-Analyzer> python src/analyzer.py
ANALYZER TEST …
"Python",
"Logistic Regression",
"Decision Trees",
"Random Forest",
"Oracle"
],
"why_relevant": "This internship demonstrates practical programming skills in Python, experience with data preprocessing and analysis, and model buildi
ng, which are foundational for a Software Engineer Trainee. The use of Oracle also shows database interaction."
},
{
"project": "Employee Promotions Prediction using Machine Learning",
"technologies": [
"Python",
"Oracle",
"Jupyter Notebook",
"Logistic Regression",
"Decision Trees",
"LightGBM"
],
"why_relevant": "This project showcases end-to-end development of a predictive model, using Python and Oracle for data handling and various ML algorith
ms, indicating strong problem-solving and programming abilities. It includes data preprocessing and model assessment, relevant to software development processes."
},
{
"project": "LLMs and AI Agents Intern",
"technologies": [
"AI Agents",
"LLMs"
],
"why_relevant": "This internship highlights the ability to develop automated task-specific workflows using advanced AI technologies, demonstrating stro
ng programming and problem-solving skills, and an aptitude for developing new systems."
}
],
master*+ 0 0 Git Graph 8 files and 0 cells to analyze
Ln 474, Col 37 Spaces: 4 UTF-8 CRLF Python Stanza venv (3.14.2)
33°C Sunny
Search
ENG IN
01:40 PM 26-09-2026



STEP 13 — Define the API

Create:

`Backend/app.py`

You should have:

Health

`GET /api/health`

Response:

```
{
  "status": "success",
  "message": "Resume JD Analyzer API is running."
}
```

Analyze

`POST /api/analyze`

Input:

```
Resume.pdf
Job_Description.pdf
```

Output:

```
{
  "status": "success",
  "matching_skills": [],
  "missing_skills": [],
  "relevant_projects": [],
  "resume_suggestions": [],
  "interview_questions": [],
  "technical_questions": [],
  "section_relevance": []
}
```

STEP 14 — Test backend

Run:

```
python app.py
```

Then test:

```
GET /api/health
```

Then upload:

```
Resume.pdf
Job_Description.pdf
```

Test `/api/analyze`.

Your backend should successfully return:

```
Resume
+
JD
+
RAG
+
Gemini
+
Analysis
```

before you start the frontend.

For the browser, only test:

Plain text

```
http://127.0.0.1:5000/api/health
```

You should get:

JSON

```
{
  "status": "success",
  "message": "Resume JD Analyzer API is running."
}
```

If you want to test `/api/analyze` from PowerShell

You can also run:

powershell

```
curl.exe -X POST "http://127.0.0.1:5000/api/analyze" `
  -F "resume=@documents/Resume.pdf" `
  -F "job_description=@documents/Job_Description.pdf" `
  -F "question=What skills are missing from my resume for this job?"
```

TO powershell you need to work with this backend project to run:

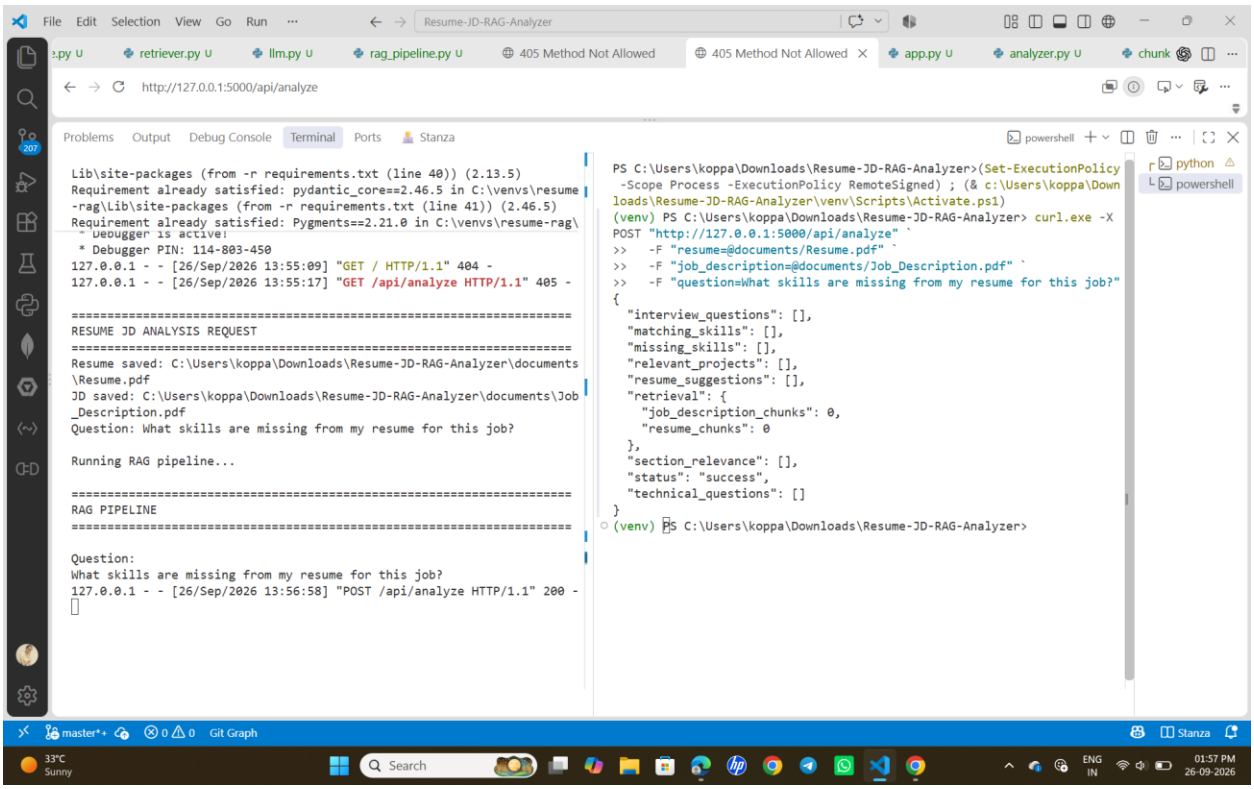
Run in terminal below 4 lines code at a time you paste in the termainla:

curl.exe -X POST "http://127.0.0.1:5000/api/analyze" `

  -F "resume=@documents/Resume.pdf" `

  -F "job_description=@documents/Job_Description.pdf" `

  -F "question=What skills are missing from my resume for this job?"



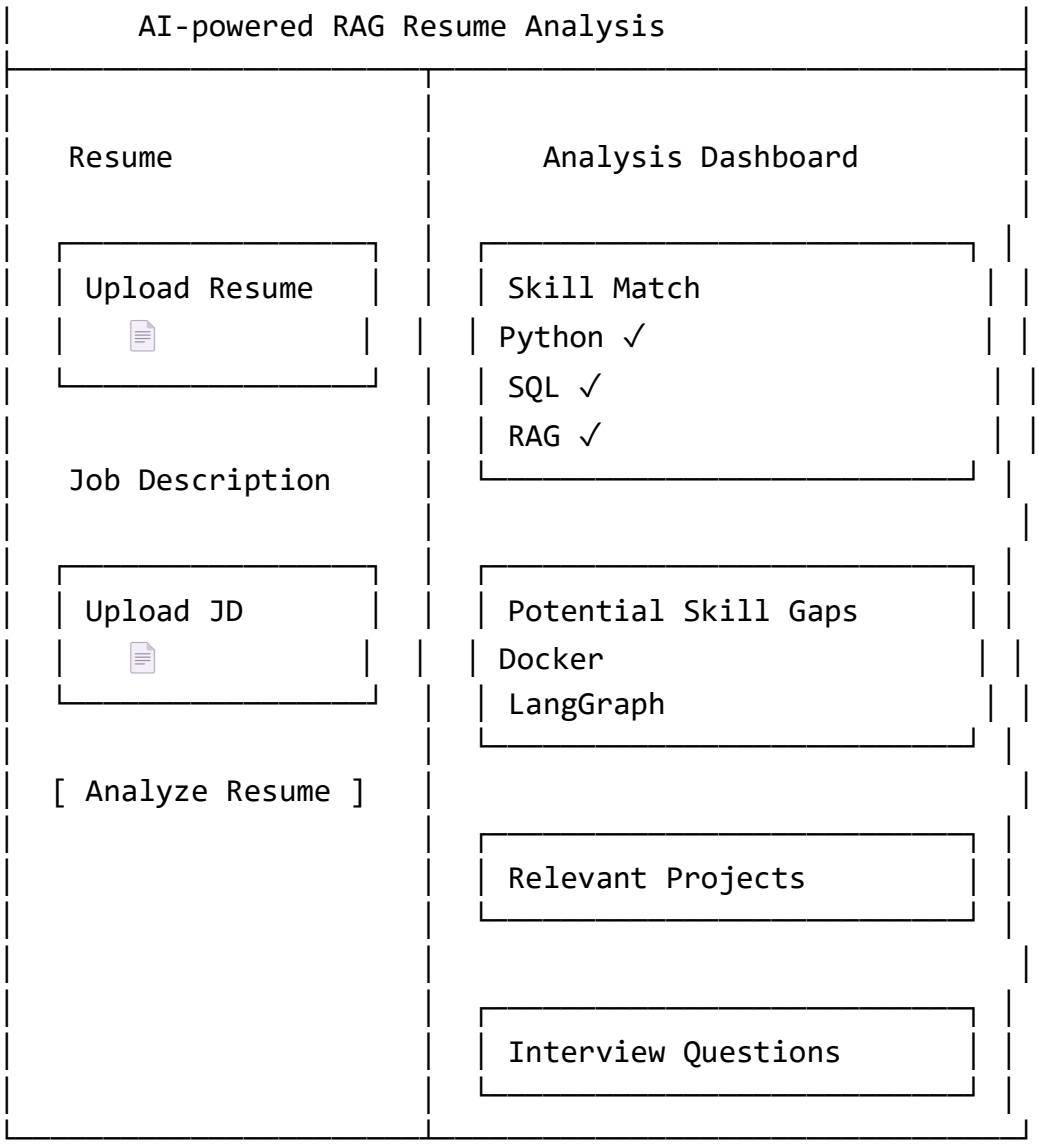
STEP 15 — Build React frontend

Create:

```
Frontend/
├── src/
│   ├── assets/
│   ├── App.jsx
│   ├── App.css
│   ├── index.css
│   └── main.jsx
├── package.json
└── vite.config.js
```

Your UI can have this structure:

```
┌──────────────────────────────────────────────────────────┐
│        💼 Resume + Job Description Analyzer              │
```



STEP 16 — Add dashboard sections

Your React application can have tabs/cards:

📊 Overview
🎯 Skill Matching
⚠️ Skill Gaps
💼 Relevant Projects
📝 Resume Suggestions

```
🎤 Interview Questions
💻 Technical Questions
📚 Sources
```

This will make the project look much more complete than simply showing one Gemini response.

STEP 17 — Add source citations

This is particularly useful for demonstrating RAG.

For every analysis section, show:

```
Source:
Resume.pdf — Page 2
```

or:

```
Source:
Job_Description.pdf — Page 1
```

For example:

```
Skill Match
————————————————————

Python ✓

Found in:
Resume.pdf — Page 1

Required by:
Job_Description.pdf — Page 1
```

STEP 18 — Add an interactive question box

At the bottom:

```
Ask something about this Resume & Job Description...

[ Type your question                          ] [➤]
```

Examples:

```
Why is my NLP project relevant to this job?

Which required skills are not mentioned in my resume?

What technical questions could be asked about my projects?

Which part of my resume should I improve for this JD?
```

The question goes through:

```
Question
 ↓
Embedding
 ↓
FAISS
 ↓
Relevant Resume/JD chunks
 ↓
Gemini
 ↓
Answer + Sources
```

STEP 19 — Final project architecture

Your completed project will look like:

```
Resume-JD-RAG-Analyzer/
│
├── Backend/
│   │
│   ├── documents/
│   │   ├── Resume.pdf
│   │   └── Job_Description.pdf
```

```
│   │
│   ├── vector_store/
│   │   ├── index.faiss
│   │   └── metadata.pkl
│   │
│   ├── src/
│   │   ├── file_loader.py
│   │   ├── chunking.py
│   │   ├── embeddings.py
│   │   ├── vector_store.py
│   │   ├── retriever.py
│   │   ├── llm.py
│   │   ├── analyzer.py
│   │   └── rag_pipeline.py
│   │
│   ├── app.py
│   ├── .env
│   ├── requirements.txt
│   └── .gitignore
│
└── Frontend/
    │
    ├── public/
    ├── src/
    │   ├── assets/
    │   ├── App.jsx
    │   ├── App.css
    │   ├── index.css
    │   └── main.jsx
    │
    ├── package.json
    └── vite.config.js
```

Your development order

Don't build everything at once. Follow this exact order:

```
STEP 1   Project Setup
   ↓
```

```
STEP 2   Python Environment
   ↓
STEP 3   Environment Variables
   ↓
STEP 4   Document Loader
   ↓
STEP 5   Chunking
   ↓
STEP 6   Embeddings
   ↓
STEP 7   FAISS Vector Store
   ↓
STEP 8   Retriever
   ↓
STEP 9   Gemini LLM
   ↓
STEP 10  Analyzer
   ↓
STEP 11  RAG Pipeline
   ↓
STEP 12  Flask API
   ↓
STEP 13  Backend Testing
   ↓
STEP 14  React Setup
   ↓
STEP 15  Upload UI
   ↓
STEP 16  Analysis Dashboard
   ↓
STEP 17  Sources
   ↓
STEP 18  Question/Chat Interface
   ↓
STEP 19  Full Integration
   ↓
STEP 20  Final Testing
```

What makes this project strong for your resume

You can honestly describe the technical architecture as:

React + Flask + RAG + Sentence Transformers + FAISS + Gemini + PDF document processing + semantic retrieval + source-aware LLM analysis.

And unlike a simple resume keyword matcher, your project demonstrates the complete flow:

document understanding → chunking → embeddings → vector search → retrieval → LLM reasoning → personalized analysis.